

Osez Noël

Application web permettant la gestion d'un calendrier de l'Avent interactif.

*Projet réalisé dans le cadre de la présentation au
Titre Professionnel Développeur Web et Web Mobile*

Présenté par Isabelle Beauregard

Sujet proposé par l'école Studi • 2024-2025



SOMMAIRE

SOMMAIRE	2
Introduction	3
Résumé du projet	4
Résumé du projet en anglais	5
Remerciements	6
Cahier des charges	7
Activité type 1	9
Développer la partie front-end d'une application web ou web mobile sécurisée	9
1. Installer et configurer son environnement de travail en fonction du projet web ou web mobile	9
2. Maquetter des interfaces utilisateur web ou web mobile	16
3. Réaliser des interfaces utilisateur statiques web ou web mobile	17
4. Développer la partie dynamique des interfaces utilisateur web ou web mobile	18
Activité type 2	21
Développer la partie back-end d'une application web ou web mobile sécurisée	21
5. Mettre en place une base de données relationnelle	21
Diagramme de cas d'utilisation	21
Modèle Conceptuel de Données	22
Modèle Logique de Données :	22
Modèle Physique de Données :	23
Création des tables :	24
6. Développer des composants d'accès aux données SQL et NoSQL	30
7. Développer des composants métiers coté serveur	35
8. Documenter le déploiement d'une application dynamique web ou web mobile.	37
Conclusion et perspectives	41
ANNEXES	42

Introduction

Je me suis intéressée au développement web il y a plusieurs années maintenant. J'avais envie d'un site web pour vendre mes créations et réalisations au crochet, mais aucun site « déjà fait » ne me correspondait vraiment. J'ai essayé des CMS comme Wix ou WordPress, mais comme ils avaient tous une base assez similaire et que je ne pouvais pas vraiment faire tout ce que je désirais, j'ai essayé d'apprendre en autodidacte sur le site OpenClassRooms, mes premiers cours de HTML5 et CSS3. Cela m'a vraiment plu et j'ai voulu aller plus loin dans une formation diplômante.

En 2019, j'ai eu la chance de pouvoir intégrer une formation POEC (Préparation Opérationnelle à l'Emploi Collective) en 6 mois grâce à Pôle Emploi où j'ai obtenu un des deux CCP pour le titre professionnel CDA. Malheureusement je n'ai pas pu continuer, devant travailler.

Je me suis inscrite à la formation DWWM chez Studi en décembre 2023 lorsqu'une collègue de travail m'a dit que j'avais une appétence pour l'informatique et que je ne devais pas abandonner mes rêves. Après la fin de mon contrat en janvier 2024, j'ai pu commencer les cours de la formation.

Je suis presque à la fin de cette formation, j'ai encore tant de choses à apprendre et je pense n'avoir jamais terminé de découvrir le monde du développement web et de l'informatique en général.

Après cette formation et avoir obtenu mon diplôme je voudrais continuer sur un Bachelor, mais si on me propose un poste à mon niveau je ne serais pas contre !

Résumé du projet

Le projet « Osez Noël » consiste à développer une application web permettant de créer et gérer des calendriers de l'Avent numériques. Les utilisateurs pourront créer un compte, personnaliser leurs calendriers, ouvrir les cases en fonction de la date, et découvrir des surprises (texte, image...). Les calendriers seront partageables via un lien unique. Une gestion des utilisateurs est prévue, permettant à Michel, le gérant, de bloquer ou supprimer des comptes ou des calendriers. L'application intègre une base de données relationnelle pour les utilisateurs et calendriers, et une structure NoSQL pour le stockage des surprises. Elle devra être déployée sur un serveur accessible au public et respecter les bonnes pratiques de développement, incluant une gestion rigoureuse du projet avec Git et un Kanban. Ce projet allie interactivité, personnalisation et accessibilité pour offrir une expérience festive et moderne.

Osez Noël a été mis en place par les formateurs de Studi pour aider les apprenants à créer une application web qui couvrirait toutes les compétences du référentiel pour le titre de D2WM.

Pour ce projet qui parle de Noël on pourrait s'attendre à y voir beaucoup de vert, de rouge et toutes autres couleurs qui rappelle cette fête. J'ai cependant pris la liberté de choisir des coloris plus orientés vers le bleu très foncé, et teinte dorée pour que les personnes atteintes de protanopie puissent distinguer à peu près toutes les couleurs utilisées, tout comme une personne à la vision normale.

Résumé du projet en anglais

“Osez Noël” project consists of developing a web application to create and manage Advent calendars digital. Users will be create an account, personalize their calendars, open boxes based on the date, and discovered surprises (text, image...). Calendars will be shared by unique link. User management is planned, allowing Michel, the manager, to block or delete accounts or calendars. The application has a relational database for users and calendars, and a NoSQL structure for storing surprises. She will be deployed on a publicly accessible server and follows good development practices including rigorous project management with Git and Trello. This project combines interactivity and personalization and accessibility to offer a festive and modern experience.

“Osez Noël” was set up by the trainers at Studi to help students to create a web application that would cover all the skills in the D2WM title framework.

For this project that talks about Christmas, we could expect to see a lot of green, red and all the other colors that remind us of this holiday. However, I took the liberty of choosing colors more oriented towards very dark blue, and the golden hue so that protanopes can distinguish almost all the colors used, just like a person with normal vision.

Remerciements

Je tiens à remercier toutes les personnes qui ont été à mes côtés pendant cette année très intense. Votre soutien, vos conseils et votre collaboration ont été inestimables.

Je remercie chaleureusement mes formateurs de Studi pour leurs précieux conseils et leur soutien tout au long de ce projet. Votre appui a été essentiel pour la réussite de ce travail.

Un très grand merci à mes camarades, en particulier à José, pour son aide, ses encouragements et les moments de partage. Ton soutien moral et technique a été très apprécié.

Je tiens également à exprimer ma profonde gratitude à mes proches, en particulier ma fille, pour qui cela a été très difficile à son âge, pour sa patience, sa compréhension et son soutien indéfectible. Ta présence à mes côtés a été une source de motivation constante.

Enfin, je remercie mes amis qui ont pris le temps de tester mes applications et de fournir des retours constructifs. Vos commentaires ont été précieux pour améliorer la qualité du produit final.

Votre aide et votre soutien ont été très précieux. Merci à tous !

Avec toute ma gratitude,

Isabelle.

Cahier des charges

I. Contexte et présentation du projet

Michel, gérant du magasin "Osez Noël", souhaite moderniser son concept de calendrier de l'Avent en proposant une application web interactive qui permettra à ses clients et amis :

- De découvrir une surprise chaque jour pendant la période de l'Avent.
- De créer et personnaliser leurs propres calendriers.
- De partager ces calendriers avec leurs proches.

L'objectif principal est de concevoir une application web simple, intuitive et accessible qui répond aux besoins fonctionnels décrits.

II. Objectifs du projet

- Proposer une expérience interactive et ludique pour la période de Noël.
- Offrir aux utilisateurs la possibilité de personnaliser et de partager leurs calendriers.
- Assurer une navigation fluide et intuitive sur tous types de supports (desktop, mobile, tablette).
- Fournir à l'admin des outils d'administration pour gérer les utilisateurs et leurs calendriers.

III. Fonctionnalités demandées

a. Affichage et navigation

- Un calendrier interactif avec 25 cases, où chaque case correspond à une phrase positive quotidienne.
- Les cases ne doivent être accessibles qu'à partir de leur date correspondante ou des dates passées (les cases futures restent verrouillées).
- Responsive design pour une compatibilité sur mobiles, tablettes et ordinateurs.

b. Gestion des surprises

- Chaque case affiche une surprise (du texte) lorsqu'elle est cliquée.
- Les surprises sont récupérées via une API ou une fonction interne en fonction de l'identifiant unique du calendrier.
- Les surprises ne sont accessibles qu'aux dates autorisées.

c. Gestion des calendriers

- Les utilisateurs peuvent :
 - S'inscrire et se connecter.
 - Créer, modifier et supprimer un calendrier.
 - Accéder à leurs calendriers via un identifiant unique.

d. Partage de calendriers

- Génération d'un lien unique basé sur l'identifiant pour partager un calendrier.

e. Administration

- Michel, en tant qu'administrateur, peut :
 - Bloquer les comptes utilisateurs en cas d'abus.
 - Supprimer des calendriers depuis un espace dédié.
 - Utiliser des filtres JavaScript pour rechercher des calendriers à supprimer.

f. Base de données

- Relationnelle (SQL) : Gestion des utilisateurs, des thèmes, et des calendriers avec les cases.
 - Table users
 - Table calendars
 - Table itemsCase

- Table themes
- Non relationnelle (NoSQL) : Stockage des statistiques du profil de l'utilisateur dans une structure JSON.
 - Table ProfileData

IV. Stack technique

Frontend

- HTML5, CSS, TailwindCSS, TypeScript (surcouche à JavaScript), React pour les composants, les hooks (useEffect, useState), les interactions et le design.

Backend

- PHP Laravel.

Base de données

- SQL : PostgreSQL pour gérer les utilisateurs, les thèmes et les calendriers avec les cases.
- NoSQL : MongoDB pour stocker le nombre de calendrier et le nombre de partage de l'utilisateur dans son profil.

Tests

- Postman : pour envoyer des requêtes http vers mon serveur API pour tester si tout fonctionne correctement.

Déploiement

- Hébergement sur mon VPS chez OVH.

V. Livrables attendus

- Lien GitHub du dépôt contenant :
 - Le code source respectant les bonnes pratiques Git :
 - Une branche principale.
 - Une branche de développement avec des fonctionnalités gérées via des branches spécifiques.
 - Scripts SQL pour la création et l'intégration des bases de données.
 - Fichier README.md avec les instructions pour déployer l'application localement.
- Lien vers l'application déployée.
- Gestion de projet via un outil Kanban (ex. : Trello, Jira) comprenant :
 - Une colonne Backlog
 - Une colonne pour les fonctionnalités en cours de développement ToDo
 - Une colonne pour les fonctionnalités terminées Done
 - Une colonne Doing

VI. Contraintes

- Respect des bonnes pratiques de développement web (sécurité, clarté du code, etc.).
- Navigation fluide sur tous types de supports.
- Déploiement sur un serveur ou une plateforme accessible publiquement.

Activité type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

1. Installer et configurer son environnement de travail en fonction du projet web ou web mobile

Pour commencer mon projet Osez Noël, j'ai utilisé WSL (Le sous-système Windows pour Linux) avec la commande "wsl --install -d debian"

J'ai pris Debian car c'est ce que j'utilise sur mon VPS chez OVH.

Ensuite j'ai créé un dossier "projects/" avec la commande "mkdir projects" puis "cd projects" pour m'y rendre et y créer mon nouveau projet. De la même manière, je crée le dossier de mon application "mkdir adventCalendar" puis cd "adventCalendar/" pour m'y rendre.

```
PS C:\Users\Isa> wsl --install -d debian
Installation : Debian GNU/Linux
Debian GNU/Linux a été installé.
Lancement de Debian GNU/Linux...
Installing, this may take a few minutes...
Please create a default UNIX user account. The username does not need to match your Windows username.
For more information visit: https://aka.ms/wslusers
Enter new UNIX username: Isabelle
adduser: Please enter a username matching the regular expression
        configured via the NAME_REGEX configuration variable. Use the
        '--allow-bad-names' option to relax this check or reconfigure
        NAME_REGEX in configuration.
Enter new UNIX username: isabelle
New password:
Retype new password:
passwd: password updated successfully
Installation successful!
isabelle@MSI:~$ mkdir projects
isabelle@MSI:~$ cd projects
isabelle@MSI:~/projects$ code .
ERROR: Failed to download the VS Code server. 'wget' not installed.
Please install wget:
Debian/Ubuntu: sudo apt-get install wget
isabelle@MSI:~/projects$ mkdir adventCalendar
isabelle@MSI:~/projects$ cd adventCalendar/
isabelle@MSI:~/projects/adventCalendar$ explorer.exe .
isabelle@MSI:~/projects/adventCalendar$
```

```
isabelle@MSI:~/projects/adventCalendar$ sudo apt update
Hit:1 http://deb.debian.org/debian bookworm InRelease
Get:2 http://security.debian.org/debian-security bookworm-security InRelease [48.0 kB]
Get:3 http://ftp.debian.org/debian bookworm-backports InRelease [59.0 kB]
Get:4 http://deb.debian.org/debian bookworm-updates InRelease [55.4 kB]
Get:5 http://deb.debian.org/debian bookworm/main Translation-en [6,109 kB]
Get:6 http://security.debian.org/debian-security bookworm-security/main amd64 Packages [216 kB]
Get:7 http://ftp.debian.org/debian bookworm-backports/main amd64 Packages [265 kB]
Get:8 http://security.debian.org/debian-security bookworm-security/main Translation-en [132 kB]
Get:9 http://ftp.debian.org/debian bookworm-backports/main Translation-en [220 kB]
Get:10 http://deb.debian.org/debian bookworm-updates/main amd64 Packages [8,856 B]
Get:11 http://deb.debian.org/debian bookworm-updates/main Translation-en [8,248 B]
Fetched 7,121 kB in 1s (8,292 kB/s)
Reading package lists... Done
Building dependency tree... Done
1 package can be upgraded. Run 'apt list --upgradable' to see it.
isabelle@MSI:~/projects/adventCalendar$ sudo apt install make
Reading package lists... Done
Building dependency tree... Done
Suggested packages:
  make-doc
The following NEW packages will be installed:
  make
0 upgraded, 1 newly installed, 0 to remove and 1 not upgraded.
Need to get 396 kB of archives.
After this operation, 1,630 kB of additional disk space will be used.
Get:1 http://deb.debian.org/debian bookworm/main amd64 make amd64 4.3-4.1 [396 kB]
Fetched 396 kB in 0s (2,922 kB/s)
Selecting previously unselected package make.
(Reading database ... 9538 files and directories currently installed.)
Preparing to unpack .../make_4.3-4.1_amd64.deb ...
Unpacking make (4.3-4.1) ...
Setting up make (4.3-4.1) ...
isabelle@MSI:~/projects/adventCalendar$ make --version
GNU Make 4.3
Built for x86_64-pc-linux-gnu
Copyright (C) 1988-2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
isabelle@MSI:~/projects/adventCalendar$
```

J'ai initialisé les fichiers Dockerfile de php et Dockerfile de nginx(*Pic.1a et 1b*), default.conf.template (*Pic.2*) Makefile (*Pic.3*), docker-compose (*Pic.4*) et les fichiers .env et .gitignore dans mon projet.

```
FROM php:8.3-fpm

RUN apt-get update \
    && apt-get install -y git \
    curl \
    unzip \
    libpng-dev \
    libjpeg-dev \
    libwebp-dev \
    libfreetype6-dev \
    libzip-dev \
    libonig-dev \
    libpq-dev \
    && docker-php-ext-configure gd --with-freetype --with-jpeg --with-webp \
    && docker-php-ext-install pdo_pgsql mbstring zip gd \
    && pecl install mongodb \
    && docker-php-ext-enable mongodb

COPY --from=composer:latest /usr/bin/composer /usr/bin/composer

RUN composer -V
```

Pic.1a

```
# Utilise Nginx basé sur Alpine
FROM nginx:alpine

# Installe bash et gettext pour envsubst
RUN apk add --no-cache bash gettext

# Copie les fichiers de template dans le conteneur
COPY default.conf.template /etc/nginx/template/default.conf.template
COPY default.prod.conf.template /etc/nginx/template/default.prod.conf.template
COPY default.api.conf.template /etc/nginx/template/default.api.conf.template

# Commande d'entrée : Choisit le bon template en fonction de l'environnement
CMD ["/bin/sh", "-c", "\
    if [ \"\$ENVIRONMENT\" = \"production\" ]; then \
        envsubst \"\$SERVER_NAME \$API_SERVER_NAME\" < /etc/nginx/template/default.prod.conf.template > \
/etc/nginx/conf.d/default.prod.conf && \
        envsubst \"\$API_SERVER_NAME\" < /etc/nginx/template/default.api.conf.template > /etc/nginx/conf.d/default.api.conf; \
    else \
        envsubst \"\$SERVER_NAME\" < /etc/nginx/template/default.conf.template > /etc/nginx/conf.d/default.conf; \
    fi; \
    exec nginx -g 'daemon off;']
```

Pic.1b

```
server {
    listen 80;
    listen [::]:80;
    server_name ${SERVER_NAME};

    root /var/www/html/public;
    index index.php;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location = /favicon.ico { access_log off; log_not_found off; }
    location = /robots.txt { access_log off; log_not_found off; }

    error_page 404 /index.php;

    location ~ \.php$ {
        fastcgi_pass advent_app:9000;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include fastcgi_params;
        fastcgi_hide_header X-Powered-By;
    }

    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }
}
```

Pic.2

```

DOCKER_COMPOSE = docker compose
PHP_CONTAINER = advent_app
NGINX_CONTAINER = advent_nginx_server
FRONT_END_CONTAINER = advent_node
ARTISAN = $(DOCKER_COMPOSE) exec $(PHP_CONTAINER) php artisan
COMPOSER = $(DOCKER_COMPOSE) exec $(PHP_CONTAINER) composer
PNPM = $(DOCKER_COMPOSE) exec $(FRONT_END_CONTAINER) pnpm
CURRENT_UID := $(shell id -u)
CURRENT_GID := $(shell id -g)

.PHONY: api client keygen migrate link cache setup_php fresh help

# Commandes par défaut
all: help

# Commandes pour Docker
up:
    @echo "🚀 Démarrage des conteneurs..."
    $(DOCKER_COMPOSE) up -d --build

down:
    @echo "🛑 Arrêt des conteneurs..."
    $(DOCKER_COMPOSE) down

restart:
    @echo "🔄 Redémarrage des conteneurs..."
    $(DOCKER_COMPOSE) down && $(DOCKER_COMPOSE) up -d

logs:
    @echo "📖 Affichage des logs Nginx..."
    $(DOCKER_COMPOSE) logs -f $(NGINX_CONTAINER)

init-api:
    @echo "Création du projet Laravel..."
    $(DOCKER_COMPOSE) exec $(PHP_CONTAINER) composer create-project --prefer-dist laravel/laravel .
    make perm

init-client:
    @echo "Création du projet Next..."
    $(DOCKER_COMPOSE) exec $(FRONT_END_CONTAINER) pnpm create vite@latest . --template react-ts
    make perm

api:
    @echo "🔧 Initialisation du projet Laravel..."
    $(COMPOSER) install
    make keygen
    make link
    make perm
    make migrate

client:
    @echo "🔧 Initialisation du projet React..."
    $(DOCKER_COMPOSE) exec -d $(FRONT_END_CONTAINER) sh -c "pnpm install && pnpm dev"

keygen:
    @echo "🔑 Génération de la clé d'application Laravel..."
    $(ARTISAN) key:generate

migrate:
    @echo "🏠 Exécution des migrations..."
    $(ARTISAN) migrate

link:
    @echo "🔗 Création d'un lien symbolique pour le local storage..."
    $(ARTISAN) storage:link

cache:
    @echo "🧹 Nettoyage et mise en cache des configurations..."
    $(ARTISAN) config:clear
    $(ARTISAN) cache:clear
    $(ARTISAN) config:cache

artisan:
    $(ARTISAN) $(cmd)

composer:
    $(COMPOSER) $(cmd)

pnpm:
    $(PNPM) $(cmd)

# Commandes pour le développement
perm:
    @echo "🔧 Configuration des permissions..."
    @sudo chown -R ${CURRENT_UID}:${CURRENT_GID} ./
    @sudo chmod -R 777 api/storage api/bootstrap/cache

fresh:
    @echo "🔄 Réinitialisation complète de la base de données..."
    $(ARTISAN) migrate:fresh --seed

# Commandes d'aide
help:
    @echo "Commandes disponibles :"
```

@echo "	make up	-> Démarrage des conteneurs Docker"
@echo "	make down	-> Arrête les conteneurs Docker"
@echo "	make restart	-> Redémarre les conteneurs Docker"
@echo "	make logs	-> Affiche les logs du conteneur Nginx"
@echo "	make init	-> Configure Laravel (install, keygen, migrate)"
@echo "	make api	-> Installe les dépendances Composer et init le projet Laravel"
@echo "	make client	-> init le projet Next"
@echo "	make keygen	-> Génère la clé d'application Laravel"
@echo "	make migrate	-> Exécute les migrations Laravel"
@echo "	make link	-> Crée un lien symbolique pour le local storage"
@echo "	make cache	-> Nettoie et met en cache les configurations"
@echo "	make fresh	-> Réinitialise la base de données"
@echo "	make perm	-> Configure les permissions des dossiers"

Pic.3

```

services:
  advent_app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: advent_php_app
    working_dir: /var/www/html
    volumes:
      - ./api:/var/www/html
    depends_on:
      - advent_postgres
    networks:
      - advent_laravel_network
    command: php-fpm

  advent_nginx:
    build:
      context: ./nginx
      dockerfile: Dockerfile
    container_name: advent_nginx_server
    volumes:
      - ./:/var/www/html
      - ./nginx/certbot/conf:/etc/letsencrypt
      - ./nginx/certbot/www:/var/www/certbot
      - ./frontend/dist:/var/www/html/dist
    ports:
      - "${APP_PORT}:80"
      - "443:443"
    depends_on:
      - advent_app
    networks:
      - advent_laravel_network
    environment:
      - SERVER_NAME=${APP_DOMAIN}
      - ENVIRONMENT=${APP_ENV}
      - API_SERVER_NAME=${APP_API_DOMAIN}

  advent_certbot:
    image: certbot/certbot:v3.1.0
    container_name: advent_certbot
    volumes:
      - ./nginx/certbot/conf:/etc/letsencrypt
      - ./nginx/certbot/www:/var/www/certbot
      - ./:/var/www/html
    networks:
      - advent_laravel_network
    depends_on:
      - advent_nginx

  advent_postgres:
    image: postgres:17
    container_name: advent_postgres_db
    environment:
      POSTGRES_DB: ${DB_DATABASE}
      POSTGRES_USER: ${DB_USERNAME}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5433:5432"
    networks:
      - advent_laravel_network

  advent_mongo:
    image: mongo:8
    container_name: advent_mongo_db
    ports:
      - "27019:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_USERNAME}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD}
    volumes:
      - mongo_data:/data/db
      - ./mongo-init.js:/docker-entrypoint-initdb.d/mongo-init.js
    networks:
      - advent_laravel_network

  advent_node:
    image: node:current-alpine3.21
    container_name: advent_node_server
    working_dir: /var/www/html
    command: sh -c "npm install -g pnpm && pnpm config set store-dir /root/.pnpm-store && tail -f /dev/null"
    volumes:
      - ./frontend:/var/www/html
    ports:
      - "4173:4173"
    networks:
      - advent_laravel_network

networks:
  advent_laravel_network:
    driver: bridge

volumes:
  postgres_data:
  mongo_data:

```

Pic.4

Création d'un nouveau repository GitHub :

Sur mon compte GitHub je crée un nouveau projet défini en mode public. Dans mon IDE PHPStorm j'utilise l'option de clone pour le repository et ainsi le récupérer sur ma machine.

Après avoir initialisé les fichiers ci-dessus et avoir installé Laravel (*make api*) ainsi que React (*make client*) ci-après. Je pourrais donc faire mon premier commit sur ma branche.

-> git pull [branche] (si changements dans le projet pour éviter les conflits)

-> git status

-> git add --all -p

-> git commit -m "init api and client project"

-> git push -u origin [branche]

```
PS Microsoft.PowerShell.Core\FileSystem::\\wsl.localhost\Debian\home\isabelle\projects\adventCalendar> git commit -m "Create Themes, Update Themes api and frontend"
[feat/themes-calendars 7b2d2d6] Create Themes, Update Themes api and frontend
21 files changed, 781 insertions(+), 169 deletions(-)
create mode 100644 api/app/Http/Controllers/Theme/DeleteThemeController.php
create mode 100644 api/app/Http/Controllers/Theme/IndexThemeController.php
create mode 100644 api/app/Http/Controllers/Theme/UpdateThemeController.php
create mode 100644 api/app/Requests/DeleteThemeFormRequest.php
create mode 100644 api/app/Requests/UpdateThemeFormRequest.php
create mode 100644 frontend/src/components/EditingThemeModal.tsx
create mode 100644 frontend/src/components/Modal.tsx
PS Microsoft.PowerShell.Core\FileSystem::\\wsl.localhost\Debian\home\isabelle\projects\adventCalendar> git status
On branch feat/themes-calendars
Your branch is ahead of 'origin/feat/themes-calendars' by 1 commit.
(use "git push" to publish your local commits)

nothing to commit, working tree clean
PS Microsoft.PowerShell.Core\FileSystem::\\wsl.localhost\Debian\home\isabelle\projects\adventCalendar> git push -u origin feat/themes-calendars
Enumerating objects: 65, done.
Counting objects: 100% (65/65), done.
Delta compression using up to 16 threads
Compressing objects: 100% (35/35), done.
Writing objects: 100% (37/37), 10.74 KiB | 366.00 KiB/s, done.
Total 37 (delta 23), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (23/23), completed with 20 local objects.
To https://github.com:Isabelle-RJ/adventCalendar.git
bc261c..7b2d2d6 feat/themes-calendars -> feat/themes-calendars
branch 'feat/themes-calendars' set up to track 'origin/feat/themes-calendars'.
PS Microsoft.PowerShell.Core\FileSystem::\\wsl.localhost\Debian\home\isabelle\projects\adventCalendar> git status
On branch feat/themes-calendars
Your branch is up to date with 'origin/feat/themes-calendars'.

nothing to commit, working tree clean
PS Microsoft.PowerShell.Core\FileSystem::\\wsl.localhost\Debian\home\isabelle\projects\adventCalendar>
```

Pic.2c

Comme il me manquait 'curl' pour réussir ma commande "make up", j'ai donc dû faire des commandes supplémentaires :

-> sudo usermod -aG docker \$USER

-> sudo apt-get install curl

-> make up

-> make init-api

Ensuite je peux lancer mes commandes pour démarrer mes conteneurs Docker, (*make up*) installer les dépendances Composer, initialiser le projet Laravel (*make init-api*). Avec la commande "make init-api" je crée mon projet Laravel et comme je l'ai créé dans le container Docker je dois lui donner les permissions de l'utilisateur sur ma machine avec la commande "make perm" (cf. Makefile). (Pic.3a & 3b)

```
isabelle@MSI: ~/projects/adv
isabelle@MSI:~/projects/adventCalendar$ make up
Démarriage des conteneurs...
docker-compose up -d --build
[+] Building 49.9s (13/13) FINISHED                                docker:default
=> [app internal] load build definition from Dockerfile            0.0s
=> => transferring dockerfile: 492B                                0.0s
=> [app internal] load metadata for docker.io/library/php:8.3-fpm  1.4s
=> [app internal] load metadata for docker.io/library/composer:latest 1.4s
=> [app auth] library/composer:pull token for registry-1.docker.io 0.0s
=> [app auth] library/php:pull token for registry-1.docker.io      0.0s
=> [app internal] load .dockerignore                               0.0s
=> => transferring context: 2B                                       0.0s
=> [app stage-0 1/4] FROM docker.io/library/php:8.3-fpm@sha256:ed962fadf621ee02ef8c760c40d5ecd58779d512c331bfea2 7.5s
=> => resolve docker.io/library/php:8.3-fpm@sha256:ed962fadf621ee02ef8c760c40d5ecd58779d512c331bfea2d614551626e4 0.0s
=> => sha256:6f4987d4078a08e6f2305c801b0e6f409a08f5132ba07a6078832a0781b4b8f7 0.1s / 0.1s
```

Pic.3a

```
[...]
=> naming to docker.io/library/adventcalendar-app:latest 0.0s
=> unpacking to docker.io/library/adventcalendar-app:latest 0.5s
=> [app] resolving provenance for metadata file 0.0s
[+] Running 8/8
✔Network adventcalendar_laravel_network Created 0.0s
✔Volume "adventcalendar_mongo_data" Created 0.0s
✔Volume "adventcalendar_postgres_data" Created 0.0s
✔Container postgres_db Started 1.0s
✔Container bun_server Started 1.0s
✔Container mongo_db Started 1.0s
✔Container php-app Started 1.2s
✔Container nginx_server Started 1.4s
isabelle@NSI:~/projects/adventCalendar$ make init-api
Création du projet Laravel...
docker-compose exec app composer create-project --prefer-dist laravel/laravel .
```

Pic. 3b

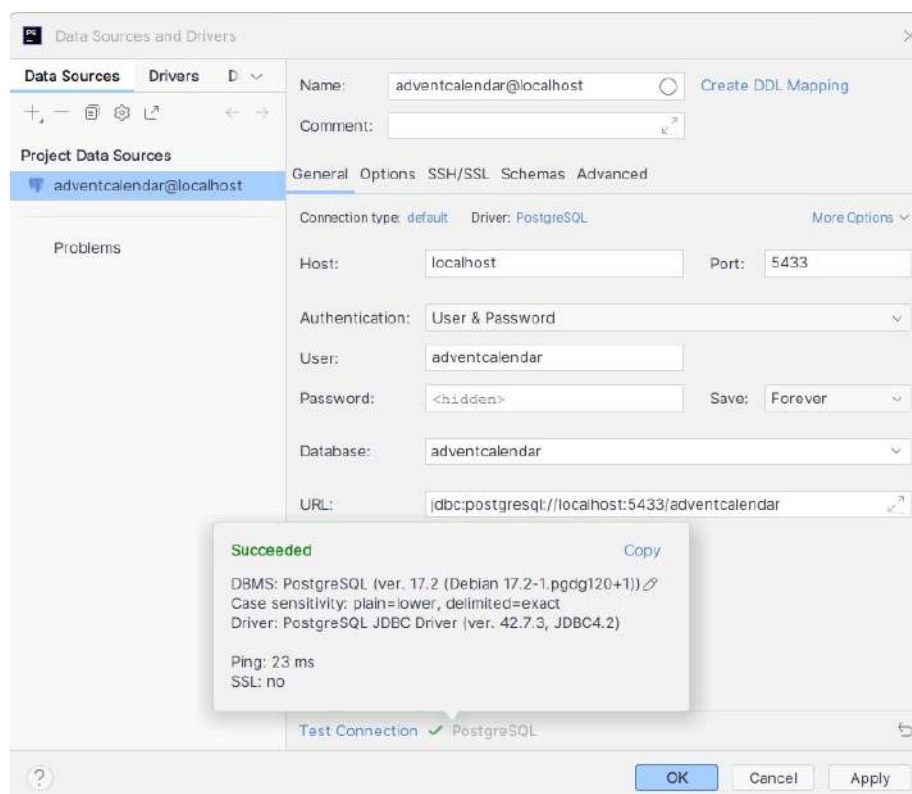
Je passe à l'édition du `./env` : je dois copier la DB_CONNEXION du fichier `./env` dans le fichier `./api/.env`

Ensuite, j'installe les dépendances avec composer-install en faisant un "make api".

Je teste la connexion à ma DB avec mon IDE PHPStorm (Pic.4)

J'initialise le projet React avec Vite (Pic.5) et installe les dépendances du projet React (Pic.6).

Pour éteindre les containers je fais la commande "make down" (puis exit pour sortir du wsl) (Pic.7)



Pic.4

```

isabelle@MSI:~/projects/adventCalendar$ make init-client
Création du projet Next...
docker-compose exec bun bun create vite . --template react-ts

Scaffolding project in /var/www/html...

Done. Now run:

    bun install
    bun run dev

make perm
make[1]: Entering directory '/home/isabelle/projects/adventCalendar'
  ✖ Configuration des permissions...
sudo chmod -R 777 api/storage api/bootstrap/cache
make[1]: Leaving directory '/home/isabelle/projects/adventCalendar'
isabelle@MSI:~/projects/adventCalendar$

```

Pic.5

```

isabelle@MSI:~/projects/adventCalendar$ make client
🔴 Initialisation du projet React...
docker-compose exec -d bun sh -c "bun install && bun dev"

```

Pic.6

```

isabelle@MSI:~/projects/adventCalendar$ exit
exit
isabelle@MSI:/mnt/c/Users/Isa$ exit
logout
PS C:\Users\Isa> msi -d debian
isabelle@MSI:/mnt/c/Users/Isa$ cd ~/projects/adventCalendar/
isabelle@MSI:~/projects/adventCalendar$ make down
🔴 Arrêt des conteneurs...
docker-compose down
[+] Running 6/6
✔Container bun_server          Removed      10.7s
✔Container nginx_server       Removed      0.7s
✔Container mongo_db           Removed      0.6s
✔Container php-app            Removed      0.5s
✔Container postgres_db        Removed      0.4s
✔Network adventcalendar_laravel_network Removed      0.2s
isabelle@MSI:~/projects/adventCalendar$

```

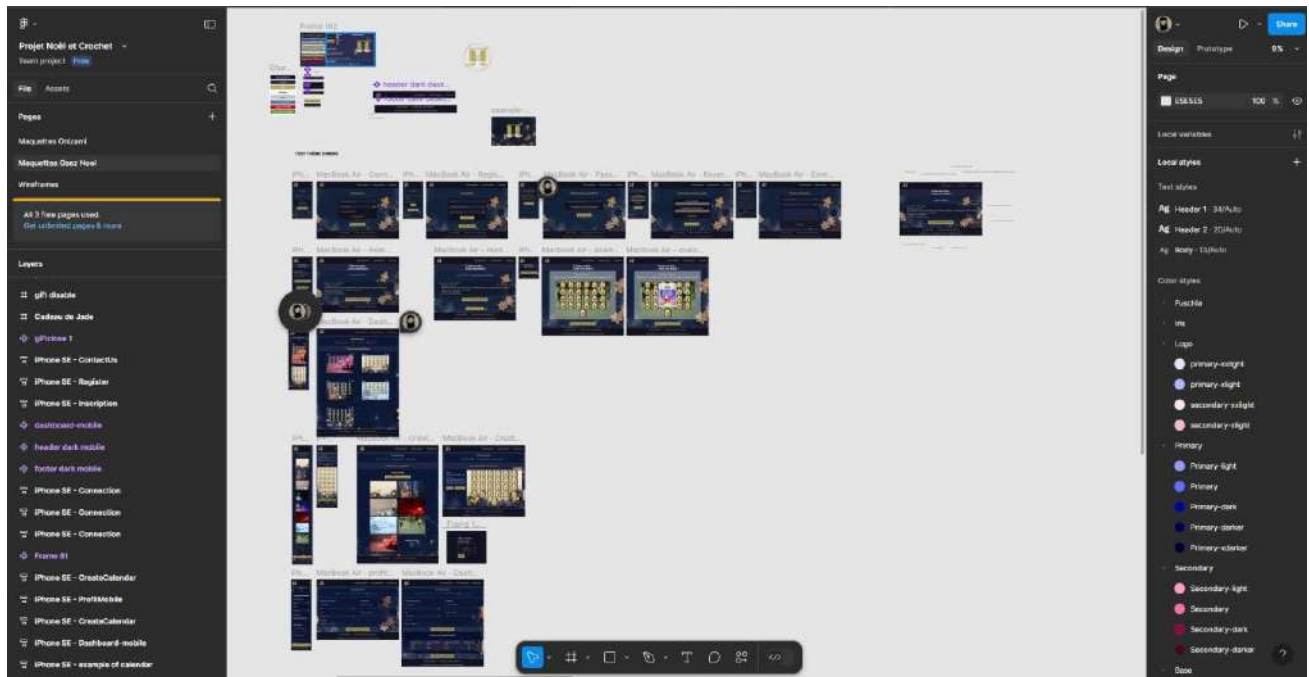
Pic.7

2. Maquetter des interfaces utilisateur web ou web mobile

Pour réaliser mes maquettes mockup et wireframe j'ai utilisé le logiciel Figma.

Je le trouve très intuitif et facile à prendre en main. Grâce à ses auto-layout et ses frames ainsi qu'aux nombreux raccourcis, les maquettes sont très rapidement réalisables, et donnent un rendu vraiment proche de ce qu'on pourrait avoir lors de l'intégration.

L'importation d'icônes également et de format SVG notamment pour le logo que j'ai réalisé sur le logiciel Canva, est vraiment pratique.



Voir la charte graphique et un extrait de ma maquette en annexes page 37

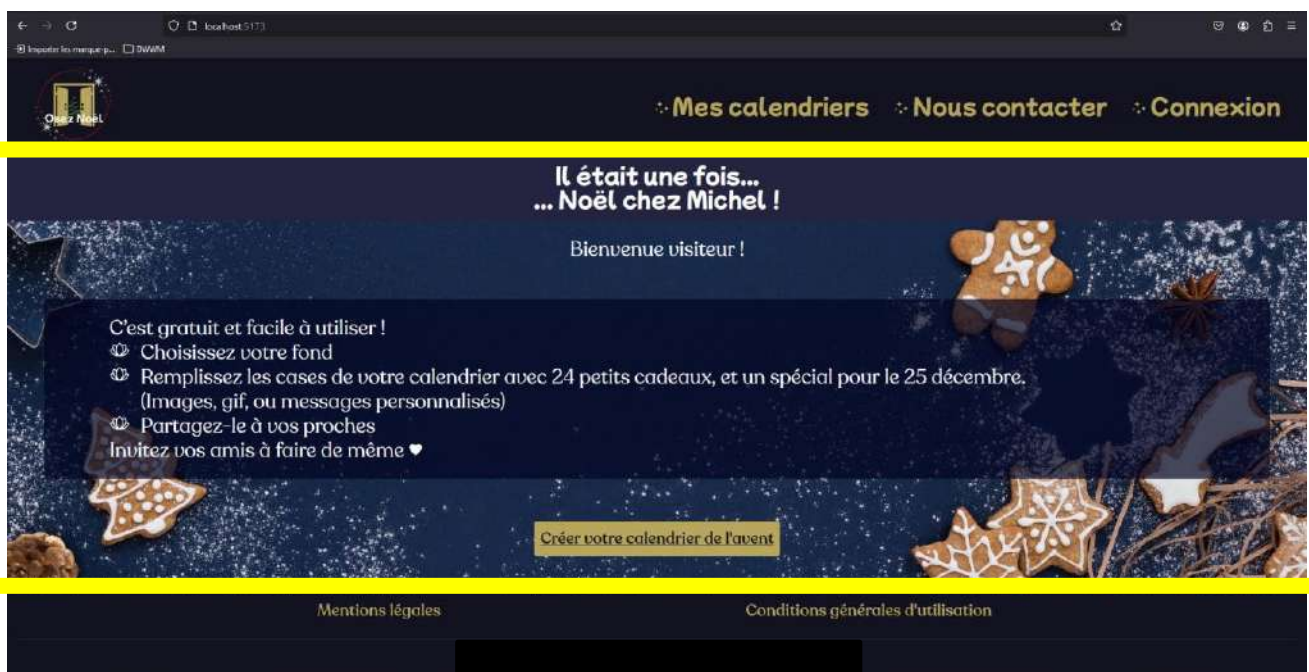
Format PDF en ligne voir ici -> [Lien cliquable](#)

3. Réaliser des interfaces utilisateur statiques web ou web mobile

Après avoir installé Tailwind CSS et fait ma première page j'ai initialisé mon projet sur GitHub.

L'extrait de code dans le fichier App.tsx fait référence à la partie encadrée en jaune ci-après :

```
frontend > src > App.tsx > ...
1  import './App.css'
2  import { Link } from 'react-router'
3  import { PiFlowerLotusLight } from "react-icons/pi"
4
5  function App() {
6    return (
7      <div>
8        <h1 className="title-home text-white text-center bg-primary-dark py-4">
9          Il était une fois... <br />
10         ... Noël chez Michel !
11        </h1>
12      </div>
13      <h2 className="text-center text-white text-3xl py-6">
14        Bienvenue visiteur !
15      </h2>
16      <div className="div-home text-white text-3xl mx-14 my-8 px-24 py-6 bg-primary-trans-dark rounded-md">
17        <p>C'est gratuit et facile à utiliser !</p>
18        <ul>
19          <li className="flex flex-row"> <PiFlowerLotusLight className="me-4" /> Choisissez votre fond</li>
20          <li className="flex flex-row"> <PiFlowerLotusLight className="me-4" /> Remplissez les cases de votre calendrier avec 24 petits cadeaux, et un
21            spécial pour le 25 décembre. <br />
22            (Images, gif, ou messages personnalisés)</li>
23          <li className="flex flex-row"> <PiFlowerLotusLight className="me-4" /> Partagez-le à vos proches</li>
24        </ul>
25        <p>Invitez vos amis à faire de même ♥</p>
26      </div>
27      <div className="div-home-submit text-center my-8 p-2">
28        <Link
29          to="/login"
30          type="submit"
31          className="btn-home-submit text-2xl hover:text-2xl underline text-center hover:text-secondary-dore hover:bg-primary-trans-dark hover:border
32            border border-secondary-dore bg-secondary-dore text-primary-x-dark p-2 rounded font-semi-bold">
33          Créer votre calendrier de l'ave
34        </Link>
35      </div>
36    )
37  }
38
39  export default App
40
41
```

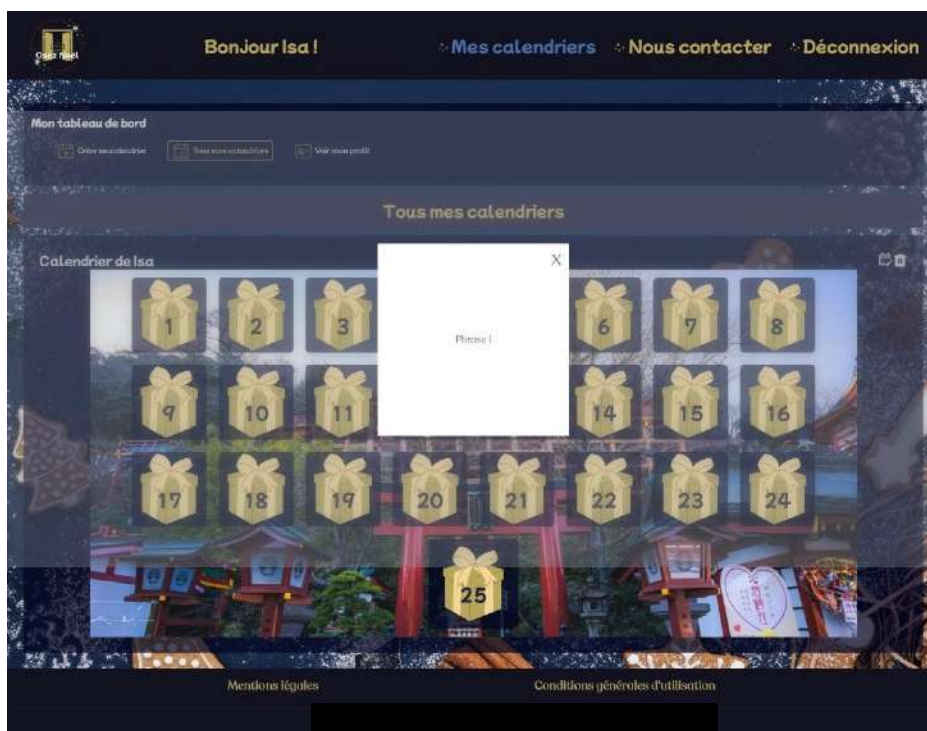


4. Développer la partie dynamique des interfaces utilisateur web ou web mobile

J'ai développé des fonctionnalités dynamiques en utilisant React, associé à Vite, afin de bénéficier d'un environnement de développement rapide et moderne.

Les interfaces dynamiques ont été conçues pour répondre aux besoins suivants :

- > Offrir une navigation intuitive et réactive.
- > Gérer des données via l'API Laravel.
- > Permettre à l'utilisateur d'interagir facilement avec les éléments de la page, tels que les cadeaux.



(captures de code avec Carbon)

ItemCase.tsx

```
import { ChangeEvent, useEffect, useRef, useState } from "react"

interface ItemCaseProps {
  id: number,
  number: number,
  gift: string,
  is_opened: boolean,
  opened_at: string,
  onDescriptionChange?: (id: number, description: string) => void,
}

export default function ItemCase({ id, number, gift, is_opened, opened_at, onDescriptionChange }:
ItemCaseProps) {
  const [open, setOpen] = useState<boolean>(false)
  const [currentDescription, setCurrentDescription] = useState<string>(gift)
  const ref = useRef<HTMLDivElement | null>(null)

  function handleClickOutside(event: MouseEvent) {
    if (ref.current && !ref.current.contains(event.target as Node)) {
      setOpen(false)
    }
  }

  useEffect(() => {
    if (open) {
      document.body.style.overflow = 'hidden'
      document.addEventListener('mousedown', handleClickOutside)
    } else {
      document.body.style.overflow = ''
    }

    return () => {
      document.body.style.overflow = ''
      document.removeEventListener('mousedown', handleClickOutside)
    }
  }, [open])

  function handleDescriptionChange(e: ChangeEvent<HTMLTextAreaElement>) {
    const newValue = e.target.value
    setCurrentDescription(newValue)
    if (onDescriptionChange) {
      onDescriptionChange(id, newValue)
    }
  }

  return (
    <> [...]
```

```

    async function uploadToDatabase(themeName: string, image: string) {
        try {
            const response = await
fetch( http://localhost:9001/api/themes/create', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
                'Accept': 'application/json',
                'X-Requested-With': 'XMLHttpRequest',
                'Authorization': `Bearer ${token}`,
            },
            credentials: 'include',
            body: JSON.stringify({
                theme_name: themeName,
                image: image,
            }),
        })

            if (!response.ok) {
                const data = await response.json()

                throw new Error(data.error ?? data.message)
            }
        } catch (error: any) {
            throw new Error(error.message)
        }
    }
}

```

SelectedTheme.tsx

La fonction asynchrone “*uploadToDatabase*” prend en paramètres le nom du thème « *themeName* » et le chemin de l’image « *image* ».

Le “try catch” permet un appel à l’API : en passant dans le try, on essaye d’envoyer la demande. Si une erreur apparaît, on va directement dans le catch qui lance un message d’erreur.

La constante “*response*” contient les données que l’API va renvoyer.

La requête est envoyée à l’URL */api/themes/create* qui est l’endpoint prévu pour créer un thème.

On envoie la demande en méthode “POST”, avec les *headers* on spécifie le type de données qu’on envoie et que l’on reçoit avec *Content-Type* et *Accept* (JSON), pour prévenir des attaques CSRF on met *'X-Requested-With': 'XMLHttpRequest'* et pour prouver que l’on a bien les droits de création sur le thème, il ne faut pas oublier le *'Authorization': Bearer \${token}* (Bearer token est la méthode d’authentification HTTP que j’utilise avec Laravel)

Body qui est le corps de la requête, convertit les données mis en paramètres, en format JSON.

- Paramètre input du fetch => URL
- Paramètres init du fetch => options du fetch

Dans le *if*, on vérifie le code de réponse http (si il n’est pas strictement égale === à 200) on récupère le message d’erreur en format JSON et on lance l’exécution d’une nouvelle erreur.

Activité type 2

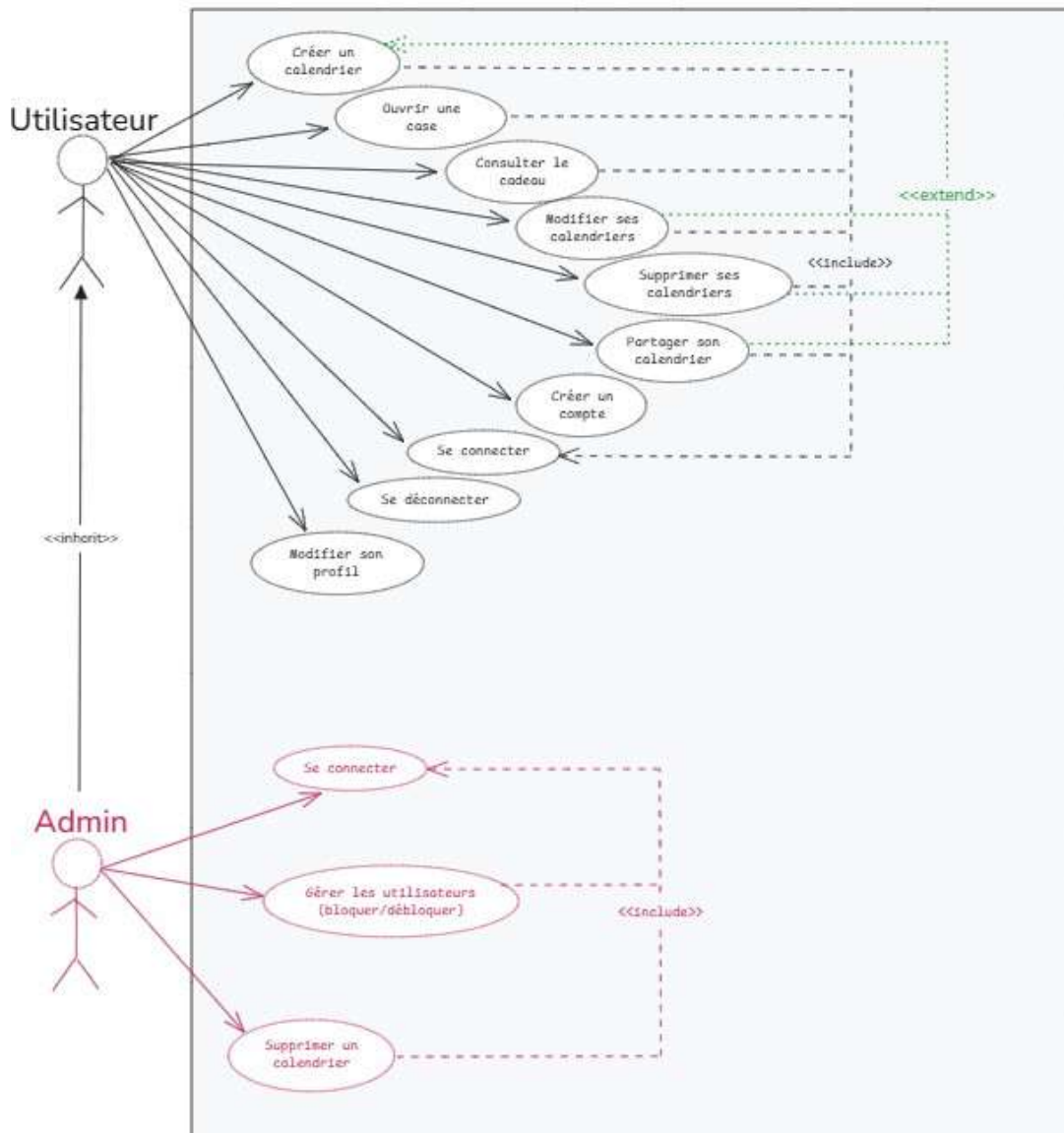
Développer la partie back-end d'une application web ou web mobile sécurisée

5. Mettre en place une base de données relationnelle

Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation est un outil clé pour garantir que le projet est bien défini, bien compris par toutes les parties, et aligné avec les besoins réels des utilisateurs. Pour réaliser le diagramme j'ai utilisé Excalidraw.

Diagramme de cas d'utilisation

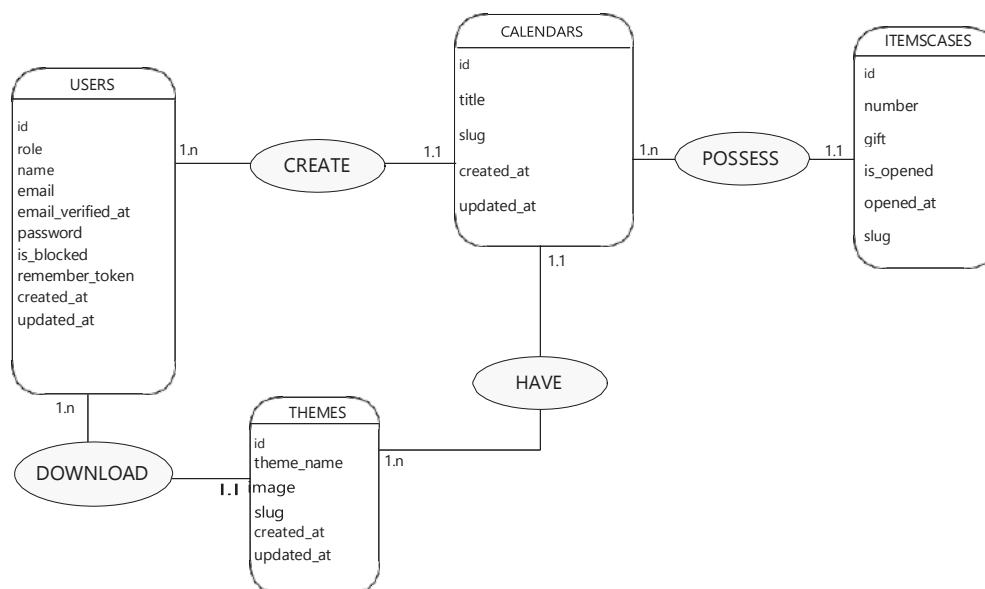


Modèle Conceptuel de Données

J'ai utilisé la méthode Merise pour réaliser les Modèles Conceptuels.

Le MCD est essentiel pour concevoir une base de données robuste. Il permet de garantir que toutes les données nécessaires sont bien prises en compte et établit une base solide pour le développement du système. Le MCD représente une vision abstraite et globale des données, indépendante des technologies spécifiques.

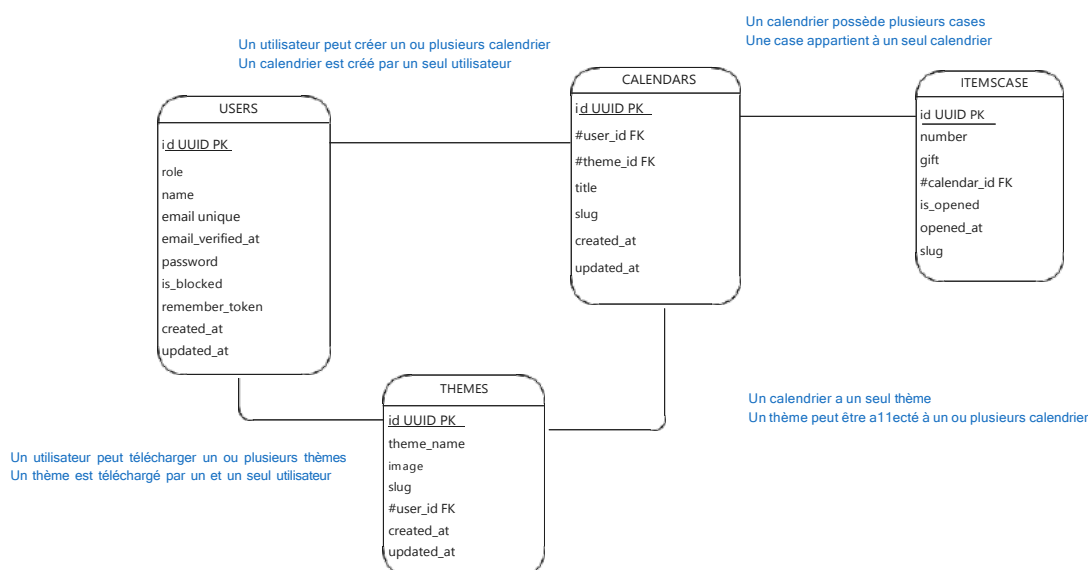
MCD OsezNoël



Modèle Logique de Données :

Le MLD constitue une étape cruciale dans la conception d'une application, car il transforme les idées abstraites du MCD en une structure claire, optimisée, et prête à être mise en œuvre. Il agit comme un pont entre la vision conceptuelle et la réalité technique. Le MLD définit les tables, les colonnes, les types de données, les clés primaires et étrangères.

MLD OsezNoël



Modèle Physique de Données :

Le MPD suit le MLD et représente la concrétisation technique de la base de données. Il inclut tous les détails nécessaires pour l'implémentation dans un Système de Gestion de Base de Données (SGBD) spécifique, tels que les types de données précis, les index, les contraintes, les partitions, et les optimisations de performance. Le MPD est dépendant du SGBD choisi et permet de générer les scripts de création de la base de données. (Les modèles ont été réalisés avec l'outil Excalidraw.)

MPD OsezNoël

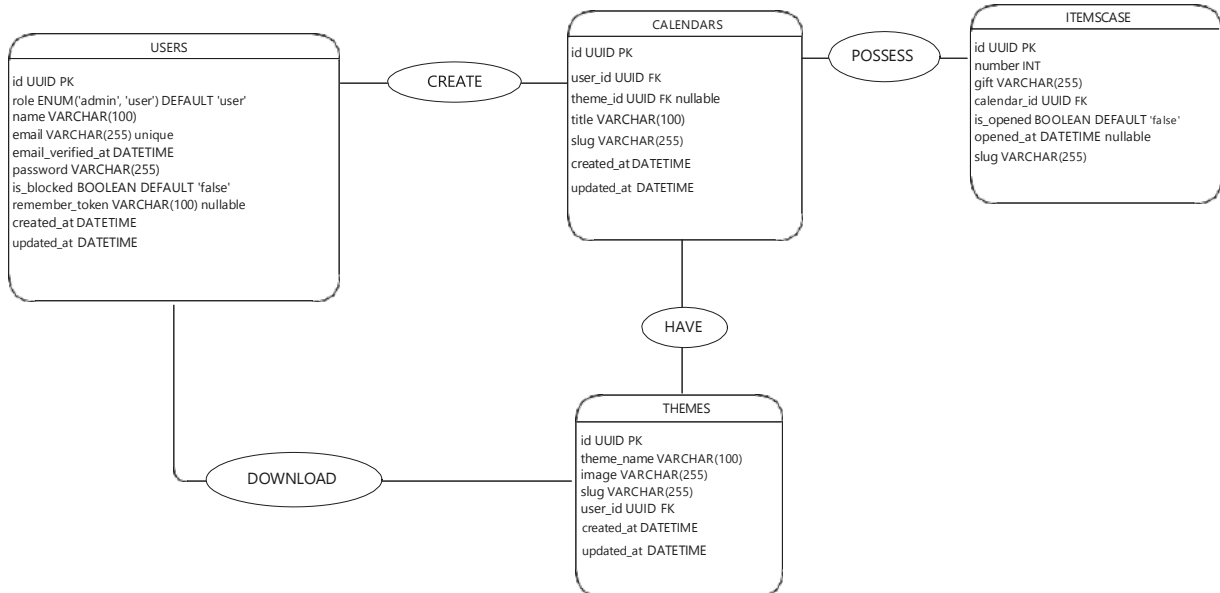
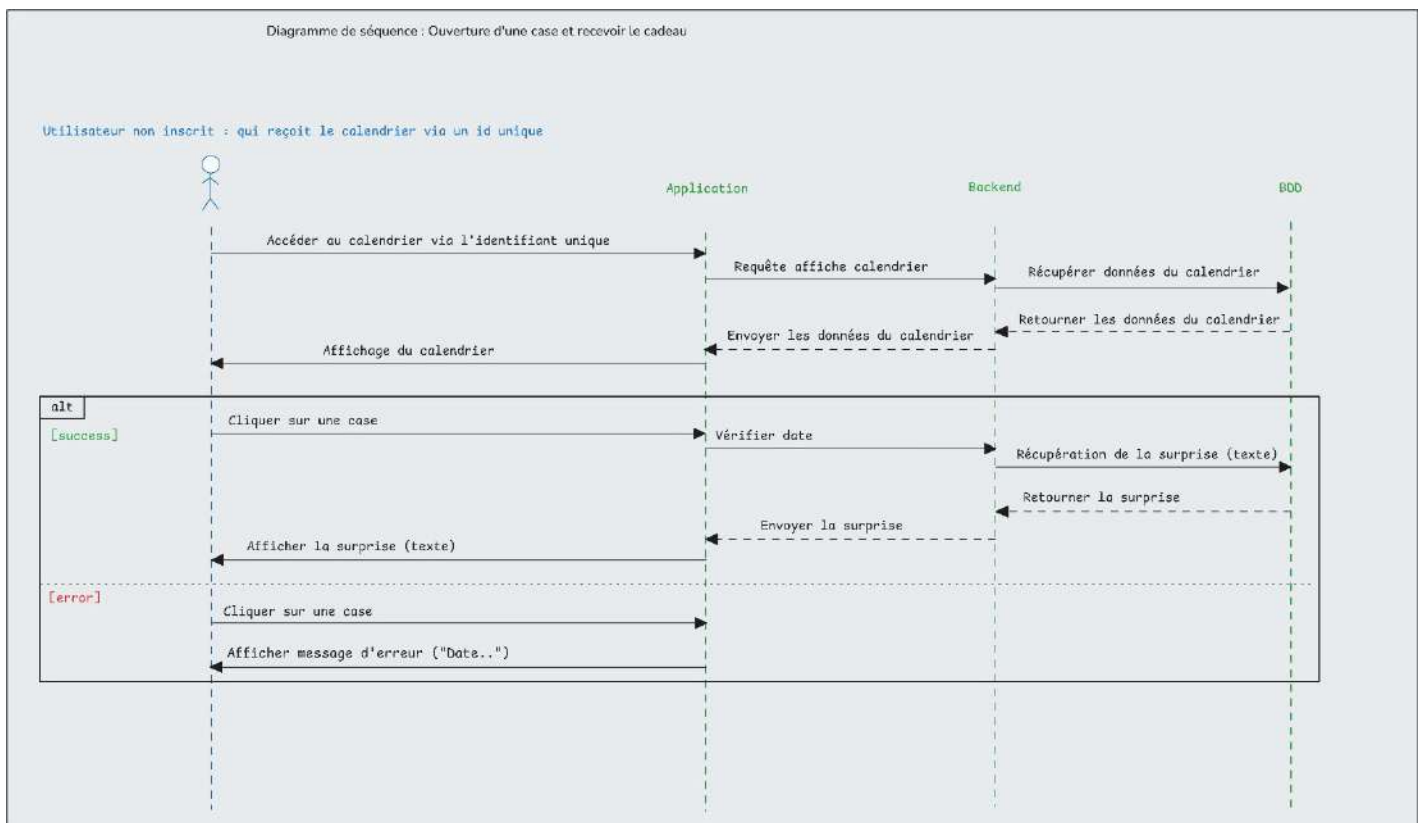


Diagramme de séquence :

-> Ouverture d'une case et recevoir le cadeau.



Création des tables :

Pour créer mes tables j'ai opté pour la solution ORM avec Eloquent de Laravel. Et pour réaliser mes migrations dans l'api Laravel j'ai dû faire les commandes suivantes :

-> `make artisan cmd="make:migration create_calendars_table"`

-> `make artisan cmd="make:migration create_itemsCases_table"`

-> `make artisan cmd="make:migration create_themes_table"`

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('calendars', function (Blueprint $table) {
            $table->uuid('id')->primary();
            $table->foreignUuid('user_id')->constrained('users')->cascadeOnDelete();
            $table->foreignUuid('theme_id')->nullable()->constrained('themes')->cascadeOnDelete();
            $table->string('title');
            $table->string('slug')->unique();
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('calendars');
    }
};
```

```

DROP DATABASE IF EXISTS adventCalendar;
CREATE DATABASE adventCalendar;
\c adventCalendar;

CREATE TABLE users
(
    id                UUID                NOT NULL PRIMARY KEY,
    role              VARCHAR(255) DEFAULT 'user' NOT NULL
        CONSTRAINT users_role_check CHECK (role IN ('admin', 'user')),
    name              VARCHAR(255)        NOT NULL,
    email             VARCHAR(255)        NOT NULL UNIQUE,
    email_verified_at TIMESTAMPTZ(0),
    password          VARCHAR(255)        NOT NULL,
    is_blocked        BOOLEAN DEFAULT FALSE NOT NULL,
    remember_token    VARCHAR(100),
    created_at        TIMESTAMPTZ(0),
    updated_at        TIMESTAMPTZ(0)
);

ALTER TABLE users OWNER TO adventcalendar;

CREATE TABLE themes
(
    id                UUID                NOT NULL PRIMARY KEY,
    theme_name        VARCHAR(255) NOT NULL,
    image             VARCHAR(255) NOT NULL,
    slug              VARCHAR(255) NOT NULL UNIQUE
);

ALTER TABLE themes OWNER TO adventcalendar;

CREATE TABLE calendars
(
    id                UUID                NOT NULL PRIMARY KEY,
    user_id           UUID                NOT NULL
        CONSTRAINT calendars_user_id_foreign REFERENCES users ON DELETE CASCADE,
    theme_id          UUID
        CONSTRAINT calendars_theme_id_foreign REFERENCES themes ON DELETE CASCADE,
    title             VARCHAR(255) NOT NULL,
    slug              VARCHAR(255) NOT NULL UNIQUE,
    created_at        TIMESTAMPTZ(0),
    updated_at        TIMESTAMPTZ(0)
);

ALTER TABLE calendars OWNER TO adventcalendar;

CREATE TABLE items_cases
(
    id                UUID                NOT NULL PRIMARY KEY,
    number            INTEGER             NOT NULL,
    gift              VARCHAR(255)        NOT NULL,
    calendar_id       UUID                NOT NULL
        CONSTRAINT items_cases_calendar_id_foreign REFERENCES calendars ON DELETE CASCADE,
    is_opened         BOOLEAN DEFAULT FALSE NOT NULL,
    opened_at         TIMESTAMPTZ(0),
    slug              VARCHAR(255)        NOT NULL
);

ALTER TABLE items_cases OWNER TO adventcalendar;

```

```

/* Exemple d'insertion dans ma base de données adventcalendar en PostgreSQL:
Activer l'extension pgcrypto pour utiliser gen_random_uuid() */
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- Insérer des utilisateurs avec un UUID généré automatiquement
INSERT INTO public.users (id, role, name, email, email_verified_at, password, is_blocked,
remember_token, created_at, updated_at)
VALUES
(gen_random_uuid(), 'user', 'Michel Dupond', 'michel.dupond@oseznoel.com', NULL,
'$2y$10$randomHash...', false, NULL, NOW(), NOW());

```

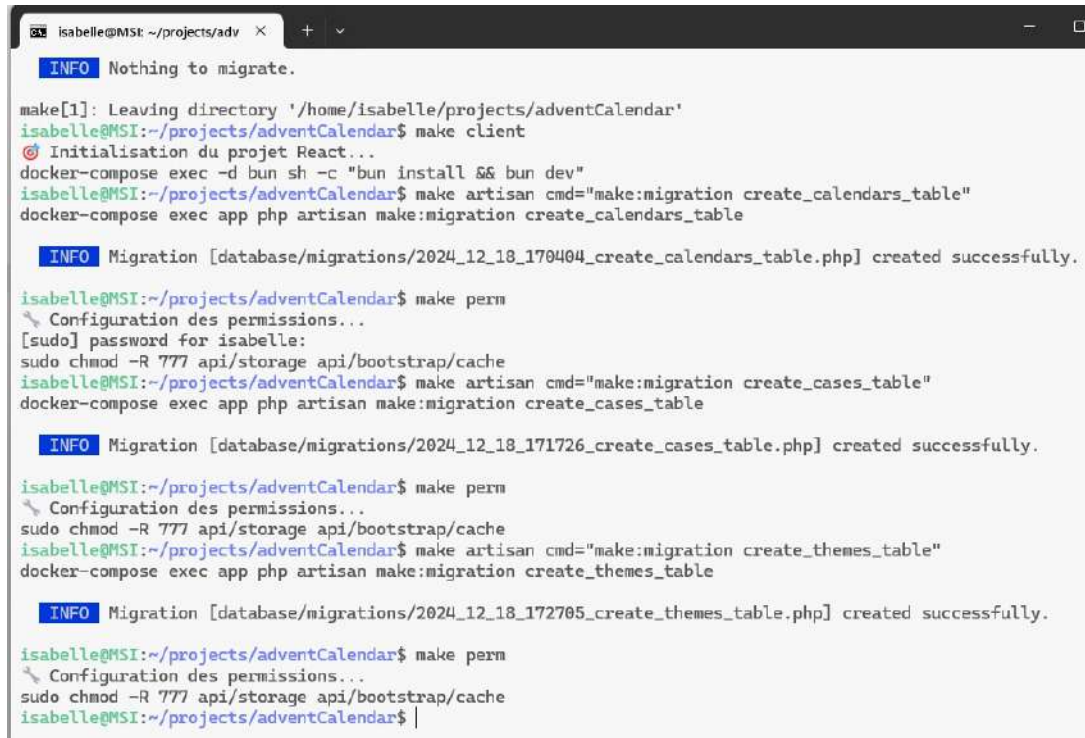
J'ai choisi d'utiliser des UUID pour plusieurs raisons :

- Plus de sécurité car ils rendent plus difficile l'incrémentation ou la prédiction d'un ID
-> Renforcement de la sécurité contre certaines attaques (*enumeration attack*)
- Unicité et portabilité : l'UUID est unique au niveau global, cela évite les conflits (fusion de BDD, synchronisation entre plusieurs serveurs, création d'entrée en BDD...)

Dans mon IDE, je suis allée dans *api/database/migrations* pour modifier mes entrées de tables créées et ainsi faire mes relations.

Puisque ma base de données tourne dans un conteneur Docker j'aurais pu me connecter à ma base de données PostgreSQL dans Docker avec des commandes dans le terminal :

`'docker exec -it postgres_db psql -U adventcalendar -d adventcalendar'`



```
isabelle@MSI: ~/projects/adv
INFO Nothing to migrate.

make[1]: Leaving directory '/home/isabelle/projects/adventCalendar'
isabelle@MSI:~/projects/adventCalendar$ make client
@ Initialisation du projet React...
docker-compose exec -d bun sh -c "bun install && bun dev"
isabelle@MSI:~/projects/adventCalendar$ make artisan cmd="make:migration create_calendars_table"
docker-compose exec app php artisan make:migration create_calendars_table

INFO Migration [database/migrations/2024_12_18_170404_create_calendars_table.php] created successfully.

isabelle@MSI:~/projects/adventCalendar$ make perm
Configuration des permissions...
[sudo] password for isabelle:
sudo chmod -R 777 api/storage api/bootstrap/cache
isabelle@MSI:~/projects/adventCalendar$ make artisan cmd="make:migration create_cases_table"
docker-compose exec app php artisan make:migration create_cases_table

INFO Migration [database/migrations/2024_12_18_171726_create_cases_table.php] created successfully.

isabelle@MSI:~/projects/adventCalendar$ make perm
Configuration des permissions...
sudo chmod -R 777 api/storage api/bootstrap/cache
isabelle@MSI:~/projects/adventCalendar$ make artisan cmd="make:migration create_themes_table"
docker-compose exec app php artisan make:migration create_themes_table

INFO Migration [database/migrations/2024_12_18_172705_create_themes_table.php] created successfully.

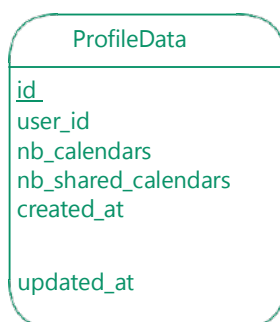
isabelle@MSI:~/projects/adventCalendar$ make perm
Configuration des permissions...
sudo chmod -R 777 api/storage api/bootstrap/cache
isabelle@MSI:~/projects/adventCalendar$ |
```

```
make[1]: Leaving directory '/home/isabelle/projects/adventCalendar'
isabelle@MSI:~/projects/adventCalendar$ make client
@ Initialisation du projet React...
docker-compose exec -d bun sh -c "bun install && bun dev"
isabelle@MSI:~/projects/adventCalendar$ docker exec -it postgres_db psql -U adventcalendar -d adventcalendar
psql (17.2 (Debian 17.2-1.pgdg120+1))
Type "help" for help.

adventcalendar=#
```

Le prompt `'adventcalendar=#'` PostgreSQL indique que la connexion est réussie.

MONGODB NoS



```
isabelle@MSI: ~/projects/adv
isabelle@MSI:~/projects/adventCalendar$ make artisan cmd="make:migration create_profile_data_table"
docker-compose exec app php artisan make:migration create_profile_data_table

INFO Migration [database/migrations/2025_02_18_163931_create_profile_data_table.php] created successfully.

isabelle@MSI:~/projects/adventCalendar$ make perm
Configuration des permissions...
[sudo] password for isabelle:
sudo chmod -R 777 api/storage api/bootstrap/cache
isabelle@MSI:~/projects/adventCalendar$
```

Migration :

```
api / database / migrations / 2025_02_18_163931_create_profile_data_table.php / class / up
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9
10     protected $connection = 'mongodb';
11
12     public function up(): void
13     {
14         Schema::create('profile_data', static function (Blueprint $collection) {
15             $collection->index('user_id');
16             $collection->integer('nb_calendars')->default(0);
17             $collection->integer('nb_shared_calendars')->default(0);
18             $collection->timestamps();
19         });
20     }
21
22     public function down(): void
23     {
24         Schema::dropIfExists('profile_data');
25     }
26 };
27
```

Je mets *\$collection* au lieu de *\$table* car c'est du Mongo.

Installation du package MongoDB pour Laravel :

```
isabelle@MSI:~/projects/adv $ make composer cmd="require mongodb/laravel-mongodb"
docker-compose exec app composer require mongodb/laravel-mongodb
./composer.json has been updated
Running composer update mongodb/laravel-mongodb
Loading composer repositories with package information
Updating dependencies
Lock file operations: 3 installs, 0 updates, 0 removals
- Locking mongodb/laravel-mongodb (5.1.1)
- Locking mongodb/mongodb (1.20.0)
- Locking symfony/polyfill-php81 (v1.31.0)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 3 installs, 0 updates, 0 removals
- Downloading symfony/polyfill-php81 (v1.31.0)
- Downloading mongodb/mongodb (1.20.0)
- Downloading mongodb/laravel-mongodb (5.1.1)
- Installing symfony/polyfill-php81 (v1.31.0): Extracting archive
- Installing mongodb/mongodb (1.20.0): Extracting archive
- Installing mongodb/laravel-mongodb (5.1.1): Extracting archive
2 package suggestions were added by new dependencies, use 'composer suggest' to see details.
Generating optimized autoload files
> Illuminate\Foundation\ComposerScripts::postAutoloadDump
> @php artisan package:discover --ansi

 INFO  Discovering packages.

intervention/image-laravel ..... DONE
laravel/pail ..... DONE
laravel/sail ..... DONE
laravel/sanctum ..... DONE
laravel/tinker ..... DONE
mongodb/laravel-mongodb ..... DONE
nesbot/carbon ..... DONE
nunomaduro/collision ..... DONE
nunomaduro/termwind ..... DONE

85 packages you are using are looking for funding.
Use the 'composer fund' command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force

 INFO  No publishable resources for tag [laravel-assets].

No security vulnerability advisories found.
Using version ^5.1 for mongodb/laravel-mongodb
isabelle@MSI:~/projects/adventCalendar$
```

Création du Model :

```
3 namespace App\Models;
4
5 > use ...
6
7
8
9
10 /**
11  * @property string $user_id
12  * @property int $nb_calendars
13  * @property int $nb_shared_calendars
14  */
15
16 no usages
17
18 class ProfileData extends Model
19 {
20     use HasApiTokens, HasUuids, HasFactory;
21
22     protected $connection = 'mongodb';
23     protected $collection = 'profile_data';
24
25     no usages
26
27     protected $fillable = [
28         'user_id',
29         'nb_calendars',
30         'nb_shared_calendars',
31     ];
32 }
```

Configuration de la base de données MongoDB encadrée en jaune :

```
104     'port' => env('DB_PORT', '1433'),
105     'database' => env('DB_DATABASE', 'laravel'),
106     'username' => env('DB_USERNAME', 'root'),
107     'password' => env('DB_PASSWORD', ''),
108     'charset' => env('DB_CHARSET', 'utf8'),
109     'prefix' => '',
110     'prefix_indexes' => true,
111     // 'encrypt' => env('DB_ENCRYPT', 'yes'),
112     // 'trust_server_certificate' => env('DB_TRUST_SERVER_CERTIFICATE', 'false'),
113 ],
114
115     'mongodb' => [
116         'driver' => 'mongodb',
117         'dsn' => env('DB_DSN'),
118         'database' => env('MONGO_DB_DATABASE', 'laravel'),
119     ],
120
121 ],
```

Configuration du .env (local)

```
30
31 MONGODB_URI=mongodb://adventcalendar:adventcalendar@mongo_db:27017/adventcalendar
32 MONGO_DB_HOST=mongo
33 MONGO_PORT=27017
34 MONGO_DB_DATABASE=adventcalendar
35 MONGO_DB_USERNAME=adventcalendar
36 MONGO_DB_PASSWORD=adventcalendar
37
```

Configuration des providers :

```
ProfileData <?php
php database 2
php providers 3 return [
.env 4     App\Providers\AppServiceProvider::class,
5     MongoDB\Laravel\MongoDBServiceProvider::class,
6 ];
7
```

J'ai utilisé MongoDB pour ma BDD NoSQL afin de stocker les statistiques des utilisateurs pour :

-> Le nombre de calendriers créer par un utilisateur.

-> Le nombre de calendriers partagés par un utilisateur.

MongoDB permet de les enregistrer et de les récupérer efficacement pour l'affichage des statistiques. (*Cf. Annexes, page 46*)

6. Développer des composants d'accès aux données SQL et NoSQL

J'ai choisi d'utiliser un contrôleur distinct pour chaque action du CRUD (Create, Read, Update, Delete – Créer, Lire, Mettre à jour, Supprimer) avec la fonction publique qui utilise la méthode `__invoke()` qui me permet de faire évoluer mon projet dans le temps.

Créer : CreateThemeController

```
<?php

namespace App\Http\Controllers\Theme;

use App\Http\Controllers\Controller;
use App\Requests\CreateThemeFormRequest;
use Illuminate\Http\JsonResponse;
use App\Models\Theme;
use Illuminate\Support\Str;

class CreateThemeController extends Controller
{
    const UPLOADS_DIRECTORY = 'uploads';

    public function __invoke(CreateThemeFormRequest $request): JsonResponse
    {
        $authenticatedUser = auth()->user();

        $theme = new Theme();
        $theme->theme_name = $request->theme_name;
        $theme->user_id = $authenticatedUser->id;
        $theme->image = $this->transformThemeName($request->theme_name);
        $theme->slug = Str::slug(pathinfo($request->theme_name, PATHINFO_FILENAME));

        $theme->save();

        return response()->json(['message' => 'Votre thème à bien été créé.']);
    }

    // Add uploads/ directory to the theme name
    private function transformThemeName(string $themeName): string
    {
        $directory = self::UPLOADS_DIRECTORY;

        return $directory . '/' . $themeName;
    }
}
```

Lire : IndexThemeController

```
<?php

namespace App\Http\Controllers\Theme;

use App\Http\Controllers\Controller;
use App\Models\Theme;
use Illuminate\Http\JsonResponse;

class IndexThemeController extends Controller
{
    public function __invoke(): JsonResponse
    {
        $themes = Theme::query()->orderByDesc('updated_at')->get();

        return response()->json(compact('themes'));
    }
}
```

Mettre à jour : UpdateThemeController

```
<?php

namespace App\Http\Controllers\Theme;

use App\Http\Controllers\Controller;
use App\Models\Theme;
use App\Requests\UpdateThemesFormRequest;
use Illuminate\Support\Facades\Storage;
use RuntimeException;
use Illuminate\Support\Str;

class UpdateThemeController extends Controller
{
    private const UPLOADS_DIRECTORY = 'uploads';

    public function __invoke(UpdateThemesFormRequest $request): void
    {
        $auth = auth()->user();

        $theme = Theme::query()->where('user_id', '=', $auth->id)->where('slug', '=', $request->slug)->first();

        if (!$theme) {
            throw new RuntimeException('Theme not found', 404);
        }

        Storage::disk('public')->delete($theme->image);

        $theme->theme_name = $request->theme_name;
        $theme->image = $this->checkPath($request->image);
        $theme->slug = Str::slug(pathinfo($request->theme_name, PATHINFO_FILENAME));

        $theme->save();
    }

    private function checkPath(string $path): string
    {
        $directory = self::UPLOADS_DIRECTORY;

        if (str_contains($path, '/uploads/')) {
            return $path;
        }

        return $directory . '/' . $path;
    }
}
```

Supprimer : DeleteThemeController

```
<?php

namespace App\Http\Controllers\Theme;

use App\Http\Controllers\Controller;
use App\Models\Theme;
use Illuminate\Support\Facades\Storage;
use RuntimeException;

class DeleteThemeController extends Controller
{
    public function __invoke(): void
    {
        $auth = auth()->user();

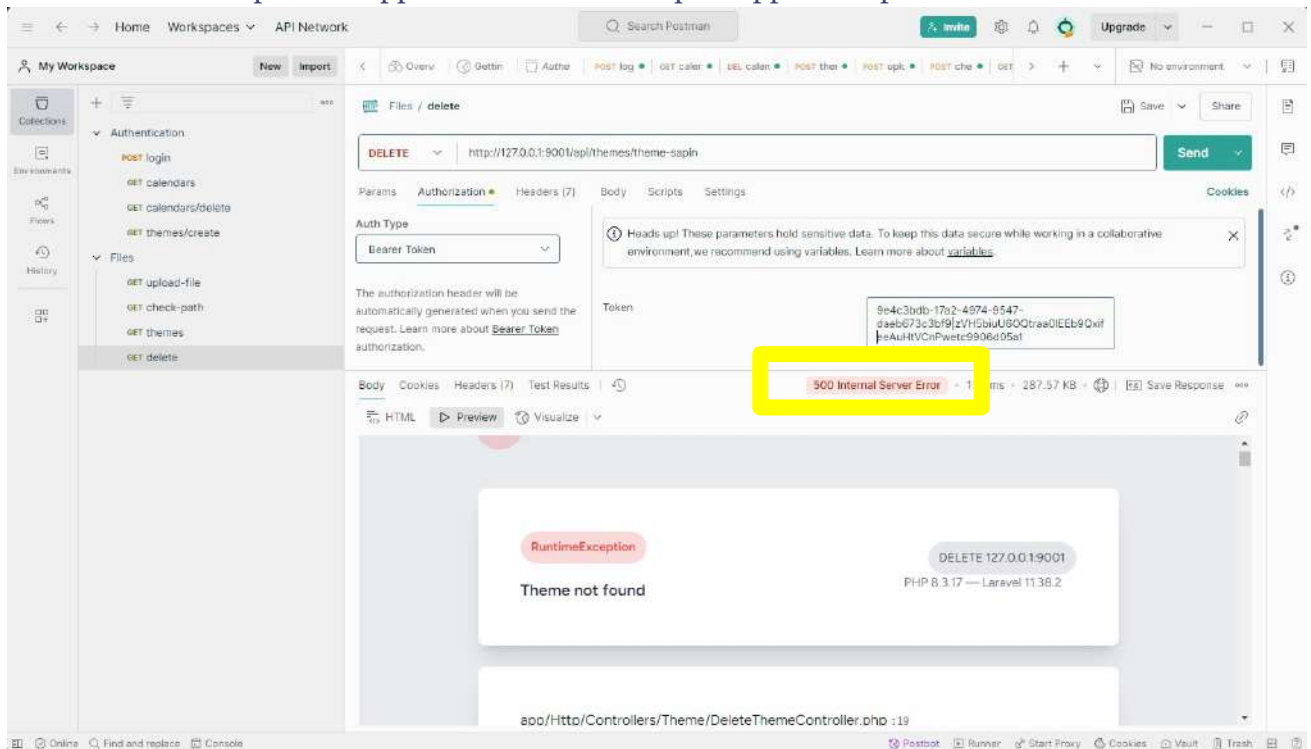
        $theme = Theme::query()->where('user_id', '=', $auth->id)->where('slug', '=', request()->slug)->first();

        if ($theme === null) {
            throw new RuntimeException('Theme not found', 404);
        }

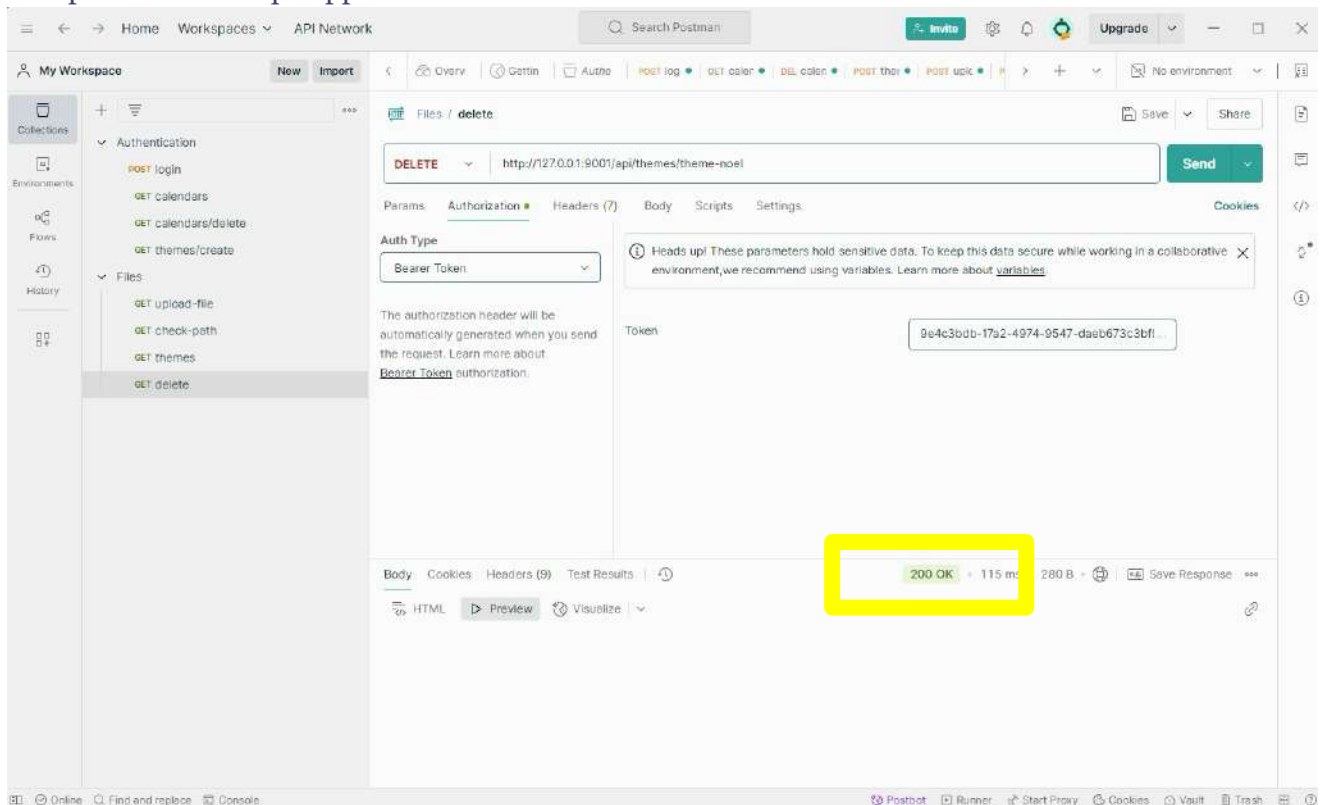
        Storage::disk('public')->delete($theme->image);

        $theme->delete();
    }
}
```

Test avec Postman pour la suppression d'un thème qui n'appartient pas à l'utilisateur connecté :



Test pour un thème qui appartient à l'utilisateur connecté :



Sécurité :

Pour sécuriser le compte administrateur de Michel, j'ai consulté la documentation de Laravel, qui recommandait d'exécuter une commande pour empêcher qu'un utilisateur puisse s'inscrire en tant qu'administrateur. J'ai donc adapté la commande initiale : `php artisan make:command SendEmails` en `make artisan cmd="make:command"`. Ensuite, j'ai modifié ma classe `AdminRegister` en ajustant la propriété `$signature`, la `$description`, ainsi que la méthode `handle`, afin de répondre aux questions nécessaires pour inscrire l'administrateur dans la base de données.

Le `make perm` est indispensable pour donner les droits au fichier créé à l'intérieur de mon conteneur Docker

```
<?php

namespace App\Console\Commands;

use App\Models\User;
use Illuminate\Console\Command;
use Illuminate\Support\Facades\Hash;

class AdminRegister extends Command
{
    /**
     * The name and signature of the console command.
     *
     * @var string
     */
    protected $signature = 'admin:register';

    /**
     * The console command description.
     *
     * @var string
     */
    protected $description = 'Admin Register Command';

    /**
     * Execute the console command.
     */
    public function handle(): int
    {
        $name = $this->ask('Quel est votre nom?');
        $email = $this->ask('Quel est votre email?');
        $password = $this->secret('Quel est votre mot de passe?');

        $validator = validator([
            [
                'name' => $name,
                'email' => $email,
                'password' => $password,
            ],
            [
                'name' => 'required|string',
                'email' => 'required|email',
                'password' => 'required|string|min:8',
            ]
        ]);

        if ($validator->fails()) {
            $this->error('Entrée invalide');
            $this->error($validator->errors());
            return self::FAILURE;
        }

        $admin = new User();
        $admin->name = $name;
        $admin->role = 'admin';
        $admin->email = $email;
        $admin->password = Hash::make($password);
        $admin->is_blocked = false;

        $admin->save();
        $this->info('Admin enregistré avec succès');

        return self::SUCCESS;
    }
}
```

```
isabelle@MSI:~/projects/adventCalendar$ make artisan cmd="make:command AdminRegister"
docker-compose exec app php artisan make:command AdminRegister
```

INFO Console command [app/Console/Commands/AdminRegister.php] created successfully.

```
isabelle@MSI:~/projects/adventCalendar$ make perm
Configuration des permissions...
[sudo] password for isabelle:
sudo chmod -R 777 api/storage api/bootstrap/cache
```

```
isabelle@MSI:~/projects/adventCalendar$ make artisan cmd="admin:register"
docker-compose exec app php artisan admin:register
```

Quel est votre nom?:

> Michel

Quel est votre email?:

> michel@oseznoel.fr

Quel est votre mot de passe?:

>

Admin enregistré avec succès

```
isabelle@MSI:~/projects/adventCalendar$
```

7. Développer des composants métiers coté serveur

```
<?php

namespace App\Services;

use Intervention\Image\Interfaces\EncodedImageInterface;
use Intervention\Image\Interfaces\ImageInterface;

class ImageOptimizerService
{
    public function __construct(private readonly ImageInterface $image)
    {}

    public function save(string $filePath): void
    {
        $this->image->save($filePath);
    }

    public function scale(int|null $width = null, int|null $height =
null): ImageInterface
    {
        return $this->image->scale($width, $height);
    }

    public function encode(int|null $quality = null):
EncodedImageInterface
    {
        return $this->image->encodeByMediaType(quality: $quality);
    }
}
```

Comme son nom l'indique cette class est un outil d'optimisation d'image. Elle peut enregistrer, redimensionner ou encoder une image

```

<?php

namespace App\Http\Controllers\File;

use App\Http\Controllers\Controller;
use Illuminate\Support\Facades\Storage;
use App\Services\ImageOptimizerService;
use App\Requests\UploadFileFormRequest;
use Illuminate\Http\JsonResponse;
use RuntimeException;
use Intervention\Image\Laravel\Facades\Image;

class UploadFilesController extends Controller
{
    public const DEFAULT_WIDTH = 1500;
    public const DEFAULT_QUALITY = 75;

    public function __invoke(UploadFileFormRequest $request):
    JsonResponse
    {
        if (!Storage::disk('public')->exists('uploads')) {
            Storage::disk('public')->makeDirectory('uploads');
            chmod(storage_path('/app/public/uploads'), 0777);
        }

        if (Storage::disk('public')->exists('uploads/' . $request-
>file('file')->getClientOriginalName())) {

            throw new RuntimeException('File already exists');
        }

        $image = Image::read($request->file('file')->getPathname());
        $optimizerService = new ImageOptimizerService($image);
        $optimizerService->scale(self::DEFAULT_WIDTH);
        $optimizerService->encode(self::DEFAULT_QUALITY);

        $optimizerService->save(storage_path('/app/public/uploads/' .
$request->file('file')->getClientOriginalName()));

        if (Storage::disk('public')->exists('uploads/' . $request-
>file('file')->getClientOriginalName())) {
            chmod(storage_path('/app/public/uploads/' . $request-
>file('file')->getClientOriginalName()), 0777);

            return response()->json([
                'message' => "L'image a bien été enregistrée",
            ]);
        }

        throw new RuntimeException("L'image n'a pas pu être
enregistrée");
    }
}

```

Le Controller *UploadFilesController* sert à enregistrer dans le Storage les fichiers qu'on a pu enregistrer / redimensionner / encoder avec la class *ImageOptimizerService* et attribue les droits au fichier grâce à la fonction native de PHP *chmod* (777 : droits sur écriture lecture et exécuter).

8. Documenter le déploiement d'une application dynamique web ou web mobile.

Mon application est déployée sur mon VPS Debian chez OVH.

Je dois créer deux sous-domaines :

- Un pour l'API : XXXX
- Un pour le frontend : XXXX

☐		0	A
☐		0	A

J'ai créé un fichier `docker-compose.yml` permettant d'orchestrer tous mes conteneurs Docker (Cf. *Annexes, page 42, Pic A : docker-compose*).

Pour une image, on retrouve les éléments suivants :

- Le build : si un Dockerfile est utilisé
- Le nom du conteneur
- Le répertoire de travail (*working directory*)
- Les volumes, si nécessaire
- Les dépendances : images dont dépend l'image principale
- Les ports exposés
- Le réseau utilisé (*network*)

Un Dockerfile est nécessaire si l'image doit être affinée (Cf. Annexes, Pic B : Dockerfile PHP).

J'utilise un template de configuration Nginx pour gérer les variables d'environnement (Cf. Annexes, Pics C et D : *Dockerfile Nginx + template local*).

Pour tester le déploiement, on modifie la variable d'environnement `APP_ENV=local` pour vérifier si le Dockerfile prend bien en compte `APP_ENV=production` défini dans le fichier `.env` et copie correctement le fichier de template `default.prod.conf.template`.

```

GNU nano /dev/shm/.../.env *
APP_DOMAIN=...
APP_API_DOMAIN=...
APP_ENV=production
APP_PORT=9001

DB_CONNECTION=pgsql
DB_HOST=advent_postgres
DB_PORT=5432
DB_DATABASE=adventcalendar
DB_USERNAME=adventcalendar
DB_PASSWORD=adventcalendar

MONGO_HOST=mongo
MONGO_PORT=27017
MONGO_DATABASE=adventcalendarmongo
MONGO_USERNAME=adventcalendarmongo
MONGO_PASSWORD=adventcalendarmongo
MONGO_AUTH_SOURCE=admin

```

Variables d'env



```

2025-03-12 15:24:43 advent_nginx_server | 2025/03/12 14:24:43 [emerg] 1#1: cannot load certificate "/etc/letsencrypt/live/localhost/fullchain.pem": BIO_new_file() failed (SSL: error:80000002:system library::No such file or directory:calling fopen(/etc/letsencrypt/live/localhost/fullchain.pem, r) error:10000080:BIO routines::no such file)
2025-03-12 15:24:43 advent_nginx_server | nginx: [emerg] cannot load certificate "/etc/letsencrypt/live/localhost/fullchain.pem": BIO_new_file() failed (SSL: error:80000002:system library::No such file or directory:calling fopen(/etc/letsencrypt/live/localhost/fullchain.pem, r) error:10000080:BIO routines::no such file)
2025-03-12 15:24:42 advent_mongo_db | {"t":"+","&data":"2025-03-12T14:24:42.827+00:00"},"c":"I","e":"CONTROL","id":"?

```

On voit bien dans les logs du conteneur nginx qu'il demande les fichiers de certificats qu'on définit dans le template.

(Cf. Annexes, Pic. E : template prod)

```

debian@vps-ac75f5a1:~/adventCalendar$ docker compose run --rm advent_certbot certonly --webroot -w /var/www/certbot -d [REDACTED]
ejurain.fr
[+] Creating 3/3
  Container advent_postgres_db Running 0.0s
  Container advent_php_app Running 0.0s
[+] Creating 3/3
  Container advent_postgres_db Running 0.0s
  Container advent_php_app Running 0.0s
  Container advent_nginx_server Running 0.0s
Saving debug log to /var/log/letsencrypt/letsencrypt.log
Requesting a certificate for [REDACTED]

[REDACTED]

NEXT STEPS:
- The certificate will need to be renewed before it expires. Certbot can automatically renew the certificate in the background, but you may need to take steps to enable that functionality. See https://certbot.org/renewal-setup for instructions.

-----
If you like Certbot, please consider supporting our work by:
  * Donating to ISRG / Let's Encrypt: https://letsencrypt.org/donate
  * Donating to EFF: https://eff.org/donate-le

```

Certificat de SSL généré.

Après avoir créé le .env nécessaire à Docker je fais la commande suivante :

```

debian@vps-ac75f5a1:~/adventCalendar$ docker compose exec advent_nginx cat /etc/nginx/conf.d/default.prod.conf
server {
    listen 80;
    listen [::]:80;
    server_name [REDACTED]

    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    listen [::]:443 ssl;
    server_name [REDACTED]

    ssl_certificate [REDACTED]
    ssl_certificate_key [REDACTED]

    root /var/www/html/public;
    index index.php;

    if ($host = isabellejurain.fr) {
        return 301 https://www.$host$request_uri;
    }

    add_header X-Frame-Options "SAMEORIGIN";
    add_header X-Content-Type-Options "nosniff";

    location / {
        proxy_pass http://127.0.0.1:5173;
    }

    location = /favicon.ico { access_log off; log_not_found off; }
    location = /robots.txt { access_log off; log_not_found off; }

    error_page 404 /index.php;

    location ~ \.php$ {
        fastcgi_pass advent_app:9000;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include fastcgi_params;
        fastcgi_hide_header X-Powered-By;
    }

    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }
}
debian@vps-ac75f5a1:~/adventCalendar$

```

```

debian@vps-ac75f5a1:~/adventCalendar$ docker compose exec advent_nginx cat /etc/nginx/conf.d/default.api.conf
server {
    listen 80;
    listen [::]:80;
    server_name [REDACTED];
    if ($host = [REDACTED]) {
        return 301 https://$host$request_uri;
    }
    return 404;
}

server {
    listen 443 ssl;
    listen [::]:443 ssl;
    server_name [REDACTED];

    ssl_certificate [REDACTED];
    ssl_certificate_key [REDACTED];

    root /var/www/html/public;
    index index.php;

    add_header X-Frame-Options "SAMEORIGIN";
    add_header X-Content-Type-Options "nosniff";

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location = /favicon.ico { access_log off; log_not_found off; }
    location = /robots.txt { access_log off; log_not_found off; }

    error_page 404 /index.php;

    location ~ \.php$ {
        fastcgi_pass advent_app:9000;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include fastcgi_params;
        fastcgi_hide_header X-Powered-By;
    }

    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }
}

```

Je lance la commande « make api » pour initialiser mon projet Laravel :

```

debian@vps-ac75f5a1:~/adventCalendar$ make api
🔗 Initialisation du projet Laravel...
docker compose exec advent_app composer install
Installing dependencies from lock file (including require-dev)
Verifying lock file contents can be installed on current platform.
Package operations: 117 installs, 0 updates, 0 removals
- Downloading doctrine/inflector (2.0.10)
- Downloading doctrine/lexer (3.0.1)
- Downloading symfony/polyfill-ctype (v1.31.0)
- Downloading webmozart/assert (1.11.0)
- Downloading dragonmantank/cron-expression (v3.4.0)
- Downloading symfony/deprecation-contracts (v3.5.1)
- Downloading psr/container (2.0.2)
- Downloading fakerphp/faker (v1.24.1)
- Downloading symfony/polyfill-php83 (v1.31.0)
- Downloading symfony/polyfill-mbstring (v1.31.0)
- Downloading symfony/http-foundation (v7.2.2)
- Downloading fruitcake/php-cors (v1.3.0)
- Downloading npr/http-message (2.0)
[...]

make[1]: Entering directory '/home/debian/adventCalendar'
🔗 Execution des migrations...
docker compose exec advent_app php artisan migrate

APPLICATION IN PRODUCTION.

Are you sure you want to run this command?
Yes

[INFO] Preparing database.
Creating migration table ..... 24.56ms DONE

[INFO] Running migrations.
0001_01_01_000000_create_users_table ..... 14.33ms DONE
0001_01_01_000001_create_cache_table ..... 5.41ms DONE
0001_01_01_000002_create_jobs_table ..... 12.35ms DONE
2024_12_18_170005_create_themes_table ..... 6.96ms DONE
2024_12_18_170404_create_calendars_table ..... 6.53ms DONE
2024_12_18_171726_create_items_cases_table ..... 4.46ms DONE
2024_12_31_123733_create_personal_access_tokens_table ..... 7.20ms DONE
2025_02_18_163931_create_profile_data_table ..... 74.89ms DONE

make[1]: Leaving directory '/home/debian/adventCalendar'
debian@vps-ac75f5a1:~/adventCalendar$

```

Ensuite, on modifie le .env du frontend, on installe avec pnpm les librairies dont mon application a besoin.

```

debian@vps-ac75f5a1:~/adventCalendar$ cd frontend/
debian@vps-ac75f5a1:~/adventCalendar/frontend$ sudo nano .env
debian@vps-ac75f5a1:~/adventCalendar/frontend$ cd ..
debian@vps-ac75f5a1:~/adventCalendar$ make pnpm cmd="install"
docker compose exec advent_node pnpm install
Lockfile is up to date, resolution step is skipped
Packages: +260
+-----+
Downloading react-icons@5.5.0: 22.22 MB/22.22 MB, done
Progress: resolved 260, reused 0, downloaded 260, added 260, done

dependencies:
+ @types/luxon 3.4.2
+ luxon 3.5.0
+ react 18.3.1
+ react-dom 18.3.1
+ react-router 7.3.0
+ react-router-dom 7.3.0

devDependencies:
+ @eslint/js 9.22.0
+ @types/react 18.3.18
+ @types/react-dom 18.3.5
+ @vitejs/plugin-react 4.3.4
+ autoprefixer 10.4.21
+ eslint 9.22.0
+ eslint-plugin-react-hooks 5.2.0
+ eslint-plugin-react-refresh 0.4.19
+ globals 15.15.0
+ postcss 8.5.3
+ react-icons 5.5.0
+ tailwindcss 3.4.17
+ typescript 5.6.3
+ typescript-eslint 8.26.1
+ vite 6.2.1

Warning
Ignored build scripts: esbuild.
Run "pnpm approve-builds" to pick which dependencies should be allowed to run scripts.

Done in 11.6s using pnpm v10.6.2
debian@vps-ac75f5a1:~/adventCalendar$

```

Puis j'ai build mon application pour que Vite fasse toute les optimisations nécessaires :

```

debian@vps-ac75f5a1:~/adventCalendar$ make pnpm cmd="build"
docker compose exec advent_node pnpm build

> html@0.0.0 build /var/www/html
> tsc -b && vite build

vite v6.2.1 building for production...
✓ 108 modules transformed.
dist/index.html                                0.50 kB | gzip:   0.33 kB
dist/assets/RecoletaAlt-Medium-C8Um2rhL.woff2 35.98 kB
dist/assets/RecoletaAlt-Thin-D-r-z94J.woff2   36.42 kB
dist/assets/RecoletaAlt-Bold-B7LSS40H.woff2   37.28 kB
dist/assets/RecoletaAlt-Light-CiPgQkd.woff2    37.58 kB
dist/assets/RecoletaAlt-Regular-Toci-Egi.woff2 37.70 kB
dist/assets/RecoletaAlt-SemiBold-Ddn8A-Nv.woff2 37.78 kB
dist/assets/RecoletaAlt-Medium-Batn5_8_.woff   51.00 kB
dist/assets/RecoletaAlt-Thin-8Im-f0wG.woff     51.20 kB
dist/assets/RecoletaAlt-Bold-CizKHFBb.woff    52.46 kB
dist/assets/RecoletaAlt-SemiBold-DWIQ0XB4.woff 52.58 kB
dist/assets/RecoletaAlt-Light-Dl4YQ5fv.woff   52.58 kB
dist/assets/RecoletaAlt-Regular-DbiGrVFm.woff 52.87 kB
dist/assets/RecoletaAlt-Thin-BIkfHySF.ttf     124.18 kB
dist/assets/RecoletaAlt-Thin-CB5rp70m.eot     124.42 kB
dist/assets/RecoletaAlt-Medium-DMeQnRtS.ttf   125.04 kB
dist/assets/RecoletaAlt-Medium-C20L2ei7.eot   125.29 kB
dist/assets/RecoletaAlt-SemiBold-D4oH70iG.ttf 125.53 kB
dist/assets/RecoletaAlt-Regular-YPJh9klv.ttf  125.55 kB
dist/assets/RecoletaAlt-SemiBold-huLedn06.eot 125.79 kB
dist/assets/RecoletaAlt-Regular-dla-G9wt.eot  125.79 kB
dist/assets/RecoletaAlt-Light-Cx4WjdB5.ttf    125.82 kB
dist/assets/RecoletaAlt-Bold-D1u0xFav.ttf     125.82 kB
dist/assets/RecoletaAlt-Bold-CTyZ9ZdU.eot     126.06 kB
dist/assets/RecoletaAlt-Light-BbusEvsI.eot     126.06 kB
dist/assets/MochiyPopOne-Regular-47k8TRUD.ttf  4,840.98 kB
dist/assets/background17-7y8xXmJs.png         36,587.62 kB
dist/assets/index-DFrVfj47.css                 22.65 kB | gzip:   5.14 kB
dist/assets/index-yXpXJ7FD.js                 380.31 kB | gzip: 122.40 kB
✓ built in 17.14s
debian@vps-ac75f5a1:~/adventCalendar$

```

Et voilà, mon application est en ligne sur [XXX](#)

Conclusion et perspectives

Qu'ai-je appris grâce à ces projets ?

J'ai appris qu'il y a une logique à suivre, la structuration est importante.

J'ai dû réfléchir à tout ce qui est base de données, créer les diagrammes, le Design, créer mes maquettes, que je dois tout faire de A à Z, sans équipe qui se partage habituellement les tâches, c'était vraiment très intense.

D'un point de vue technique en termes d'applications(web) :

J'ai appris les bases du code, la gestion de la BDD, la rigueur, et l'organisation.

Qu'ai-je appris sur moi ?

J'ai découvert que je suis plus patiente que je ne l'imaginais, et plus persévérante. Ces projets m'ont permis de sortir de ma zone de confort, et de chercher activement les réponses à mes questions.

Gestion du temps

J'ai souvent été en retard par rapport aux autres, même si je ne peux pas affirmer que c'était facile pour eux. Cependant, certains ont assimilé plus rapidement les théories et la pratique. Cela m'a appris à rester motivée malgré les difficultés et à travailler à mon rythme pour atteindre mes objectifs.

Ce que j'ai aimé le plus ?

La création des maquettes mockup et l'intégration front-end.

Ce que j'ai trouvé particulièrement difficile ?

Rédiger les dossiers et faire le diaporama, malgré mes nombreuses prises de notes sur Obsidian.

Le backend a également été un défi, notamment avec les FormRequest de Laravel, dont la documentation propose des exemples longs et parfois éloignés de mes besoins.

Vers quoi j'ai envie de m'orienter ?

Je souhaite continuer vers un Bac+3/+4 Concepteur développeur d'applications, ou Concepteur développeur Front-end en présentiel ou de préférence en alternance pour acquérir davantage d'expérience au sein d'une équipe.

ANNEXES

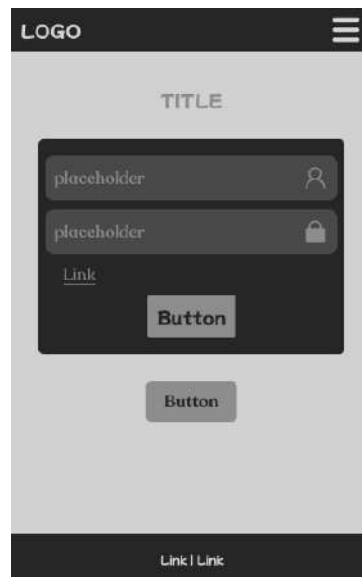
Voici la charte graphique :

Polices d'écriture, mon logo, les icônes, exemples de boutons, les couleurs utilisées et les cadeaux en format SVG (ouvert et fermé) pour les calendriers :

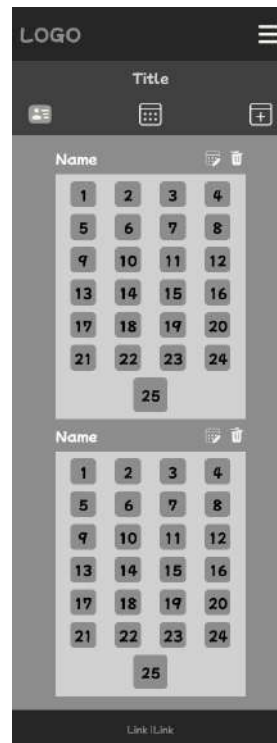
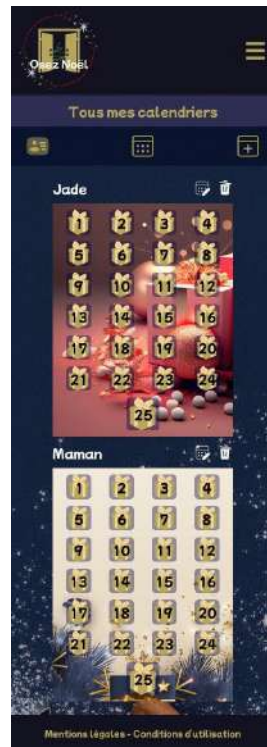


Et quelques exemples de la maquette :

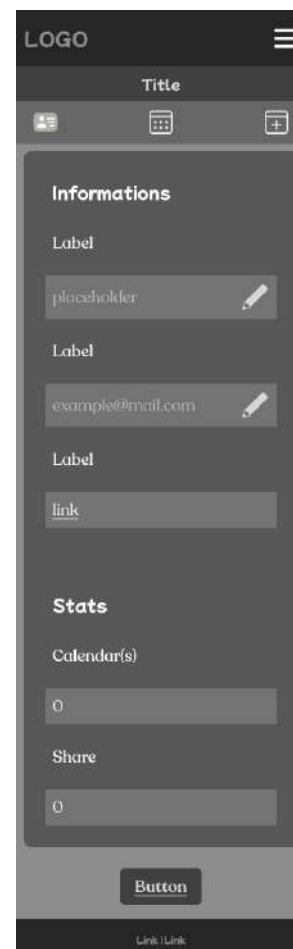
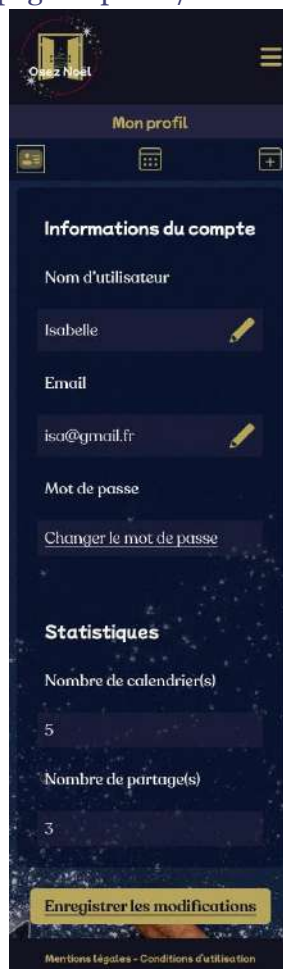
Mockup mobile / Wireframe mobile page de connexion :



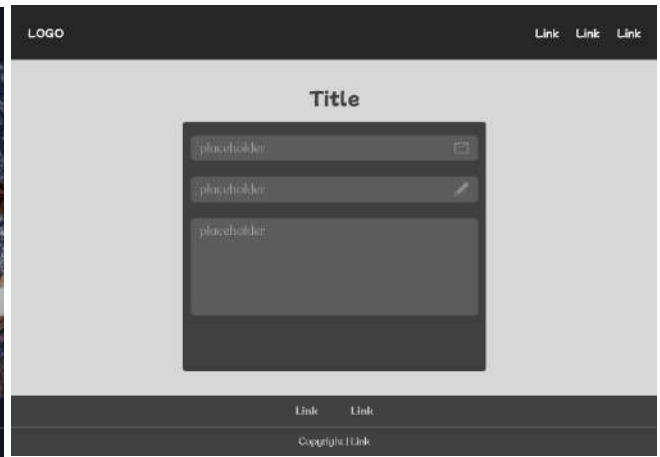
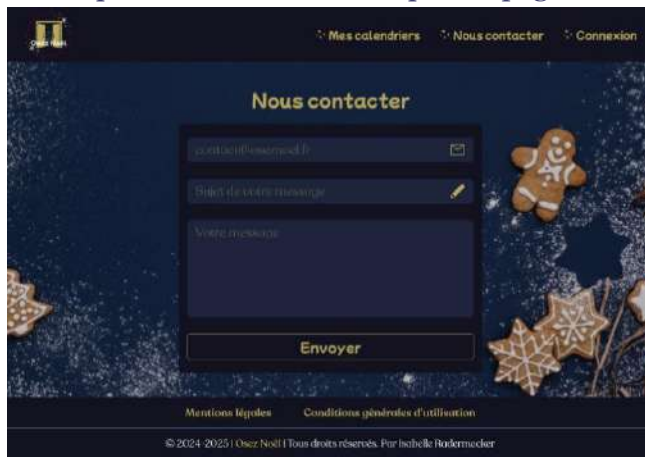
Mockup mobile / Wireframe mobile page Tableau de bord :



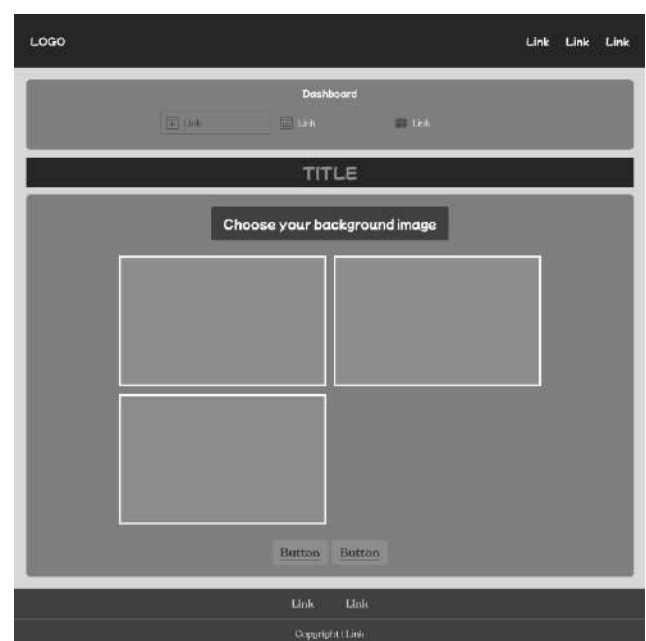
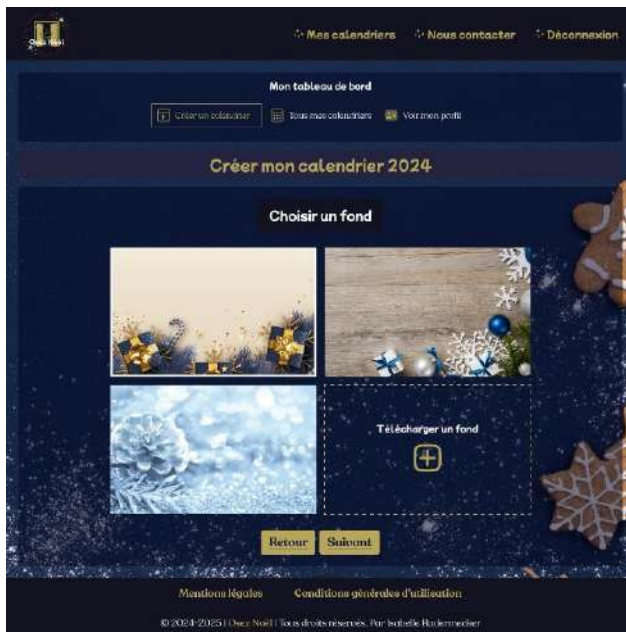
Mockup mobile page de profil / Wireframe mobile page de profil



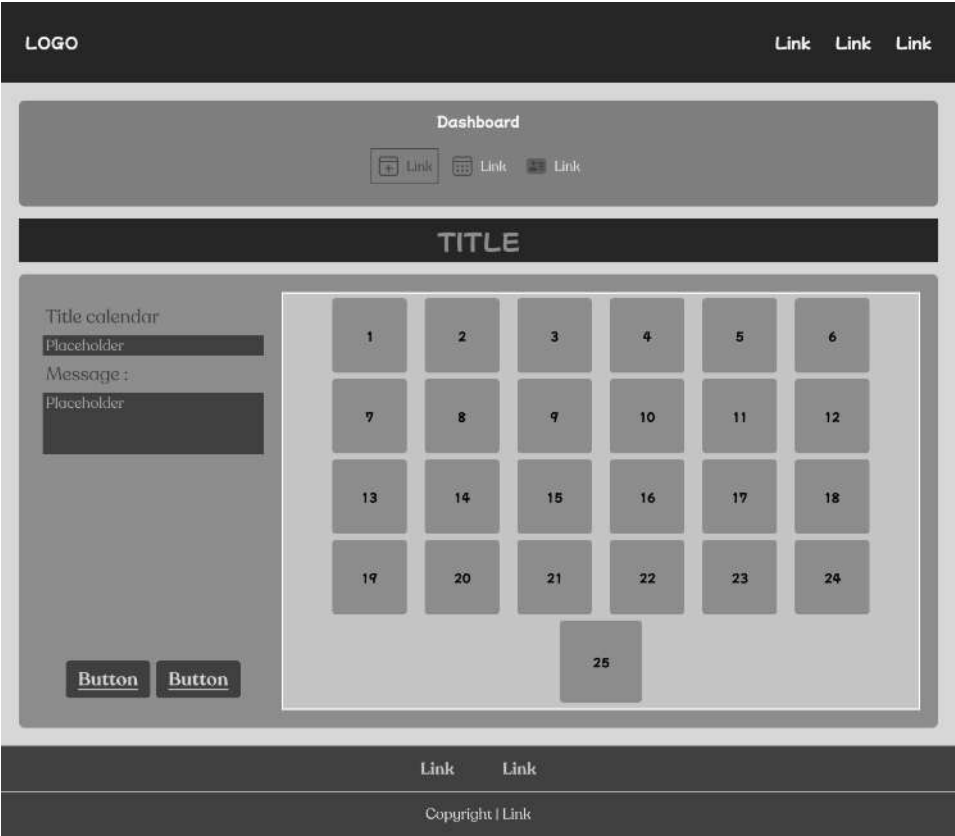
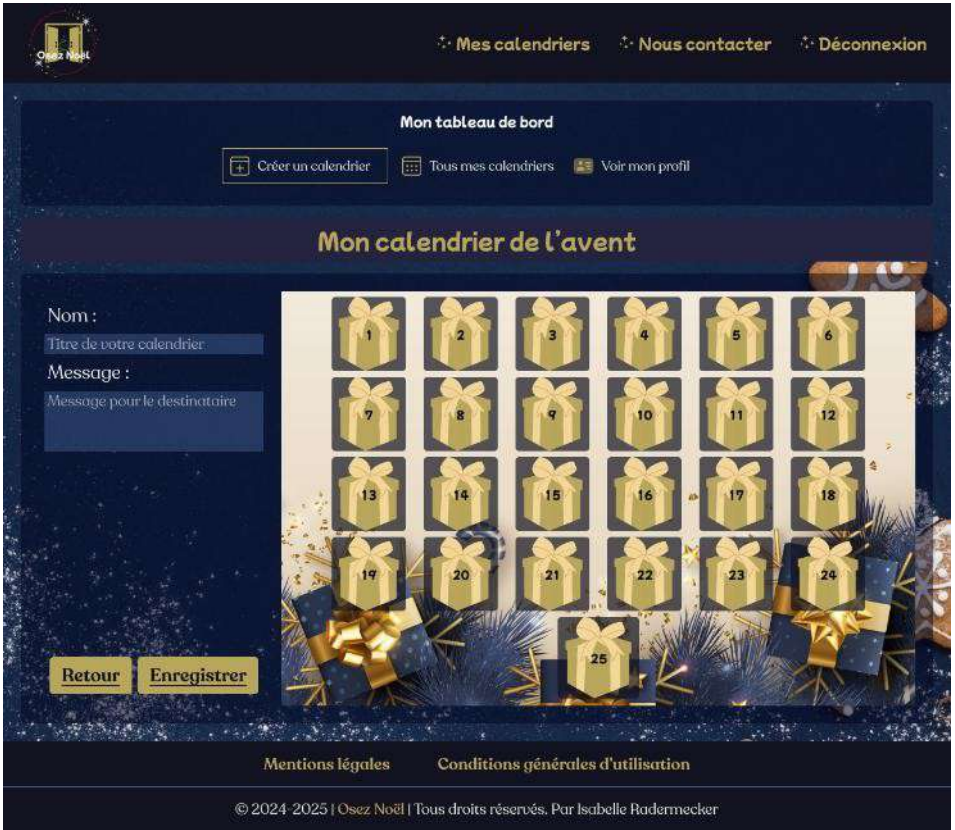
Mockup et le wireframe desktop de la page de contact :

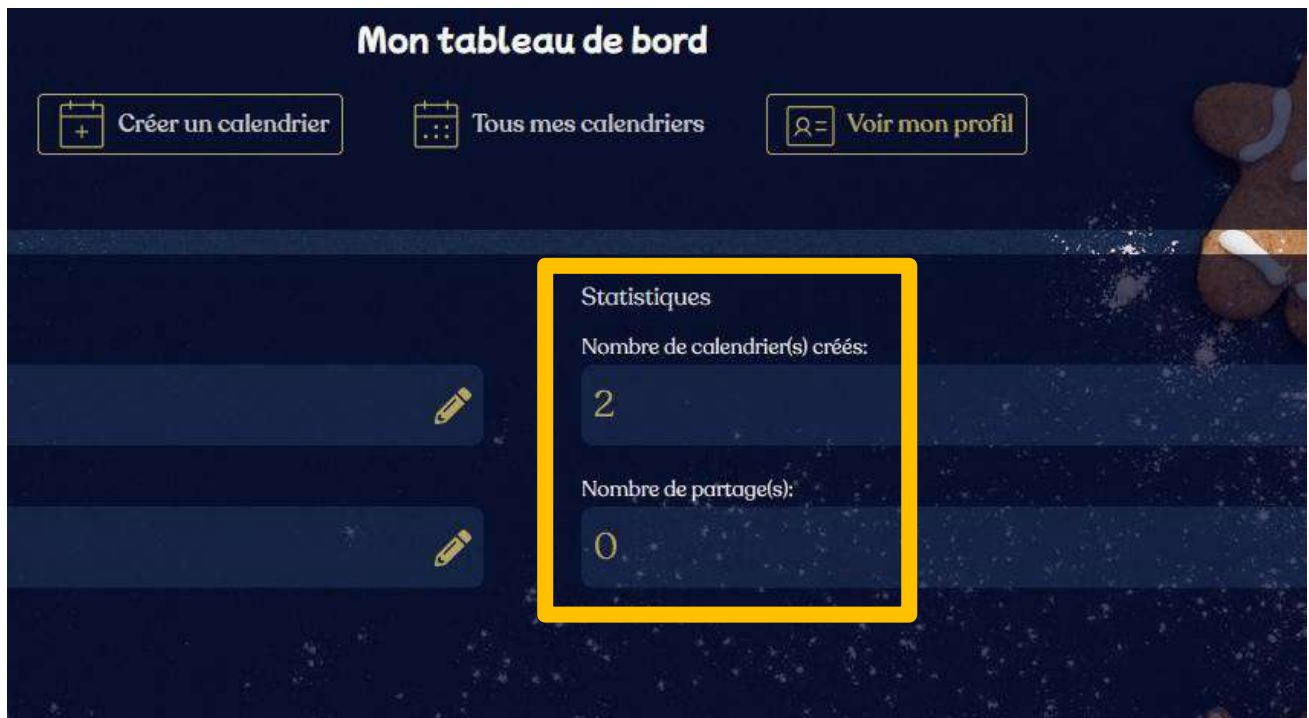


Mockup et Wireframe Desktop : page créer un calendrier (fond)



Et pour finir, les mockup et Wireframe Desktop : page créer un calendrier (cases)





```

services:
  advent_app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: advent_php_app
    working_dir: /var/www/html
    volumes:
      - ./api:/var/www/html
    depends_on:
      - advent_postgres
    networks:
      - advent_laravel_network
    command: php-fpm

  advent_nginx:
    build:
      context: ./nginx
      dockerfile: Dockerfile
    container_name: advent_nginx_server
    volumes:
      - ./:/var/www/html
      - ./nginx/certbot/conf:/etc/letsencrypt
      - ./nginx/certbot/www:/var/www/certbot
      - ./frontend/dist:/var/www/html/dist
    ports:
      - "${APP_PORT}:80"
      - "443:443"
    depends_on:
      - advent_app
    networks:
      - advent_laravel_network
    environment:
      - SERVER_NAME=${APP_DOMAIN}
      - ENVIRONMENT=${APP_ENV}
      - API_SERVER_NAME=${APP_API_DOMAIN}

  advent_certbot:
    image: certbot/certbot:v3.1.0
    container_name: advent_certbot
    volumes:
      - ./nginx/certbot/conf:/etc/letsencrypt
      - ./nginx/certbot/www:/var/www/certbot
      - ./:/var/www/html
    networks:
      - advent_laravel_network
    depends_on:
      - advent_nginx

  advent_postgres:
    image: postgres:17
    container_name: advent_postgres_db
    environment:
      POSTGRES_DB: ${DB_DATABASE}
      POSTGRES_USER: ${DB_USERNAME}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5433:5432"
    networks:
      - advent_laravel_network

  advent_mongo:
    image: mongo:8
    container_name: advent_mongo_db
    ports:
      - "27019:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_USERNAME}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD}
    volumes:
      - mongo_data:/data/db
      - ./mongo-init.js:/docker-entrypoint-initdb.d/mongo-init.js
    networks:
      - advent_laravel_network

  advent_node:
    image: node:current-alpine3.21
    container_name: advent_node_server
    working_dir: /var/www/html
    command: sh -c "npm install -g pnpm && pnpm config set store-dir /root/.pnpm-store && tail -f /dev/null"
    volumes:
      - ./frontend:/var/www/html
    ports:
      - "4173:4173"
    networks:
      - advent_laravel_network

networks:
  advent_laravel_network:
    driver: bridge

volumes:
  postgres_data:
  mongo_data:

```

Dockerfile (PHP) :

Pic B

```
FROM php:8.3-fpm

RUN apt-get update \
  && apt-get install -y git \
  curl \
  unzip \
  libpng-dev \
  libjpeg-dev \
  libwebp-dev \
  libfreetype6-dev \
  libzip-dev \
  libonig-dev \
  libpq-dev \
  && docker-php-ext-configure gd --with-freetype --with-jpeg --with-webp \
  && docker-php-ext-install pdo_pgsql mbstring zip gd \
  && pecl install mongodb \
  && docker-php-ext-enable mongodb

COPY --from=composer:latest /usr/bin/composer /usr/bin/composer

RUN composer -V
```

Dockerfile (nginx) :

Pic C

```
# Utilise Nginx basé sur Alpine
FROM nginx:alpine

# Installe bash et gettext pour envsubst
RUN apk add --no-cache bash gettext

# Copie les fichiers de template dans le conteneur
COPY default.conf.template /etc/nginx/template/default.conf.template
COPY default.prod.conf.template /etc/nginx/template/default.prod.conf.template
COPY default.api.conf.template /etc/nginx/template/default.api.conf.template

# Commande d'entrée : Choisit le bon template en fonction de l'environnement
CMD ["/bin/sh", "-c", "\
  if [ \"$ENVIRONMENT\" = \"production\" ]; then \
    envsubst '$$SERVER_NAME $$API_SERVER_NAME' < /etc/nginx/template/default.prod.conf.template > \
/etc/nginx/conf.d/default.prod.conf && \
    envsubst '$$API_SERVER_NAME' < /etc/nginx/template/default.api.conf.template > /etc/nginx/conf.d/default.api.conf; \
  else \
    envsubst '$$SERVER_NAME' < /etc/nginx/template/default.conf.template > /etc/nginx/conf.d/default.conf; \
  fi; \
  exec nginx -g 'daemon off;']
```

Default.conf.template local :

Pic D

```
server {
    listen 80;
    listen [::]:80;
    server_name ${SERVER_NAME};

    root /var/www/html/public;
    index index.php;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location = /favicon.ico { access_log off; log_not_found off; }
    location = /robots.txt { access_log off; log_not_found off; }

    error_page 404 /index.php;

    location ~ \.php$ {
        fastcgi_pass advent_app:9000;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include fastcgi_params;
        fastcgi_hide_header X-Powered-By;
    }

    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }
}
```

Default.prod.template :

Pic E

```
server {
    listen 80;
    listen [::]:80;
    server_name oseznoel.isabellejurain.fr;

    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    listen [::]:443 ssl;
    server_name ${SERVER_NAME};

    ssl_certificate /etc/letsencrypt/live/${API_SERVER_NAME}/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/${API_SERVER_NAME}/privkey.pem;

    root /var/www/html/dist;
    index index.html;

    # Gestion du routage côté client pour React
    location / {
        try_files $uri /index.html;
    }

    location /assets/ {
        expires 6M;
        access_log off;
    }

    # Rediriger les erreurs 404 vers index.html (React SPA)
    error_page 404 /index.html;
}
```